arXiv:2407.08608v2 [cs.LG] 12 Jul 2024

# FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision

Jay Shah*[1], Ganesh Bikshandi*[1], Ying Zhang[2], Vijay Thakkar[3,4], Pradeep Ramani[3], and Tri Dao[5,6]

[1]Colfax Research [2]Meta [3]NVIDIA [4]Georgia Tech [5]Princeton University [6]Together AI

{jayhshah,ganesh}@colfax-intl.com, yingz@meta.com, {vithakkar,prraman}@nvidia.com, tri@tridao.me

July 16, 2024

**Abstract**

Attention, as a core layer of the ubiquitous Transformer architecture, is the bottleneck for large language models and long-context applications. FlashAttention elaborated an approach to speed up attention on GPUs through minimizing memory reads/writes. However, it has yet to take advantage of new capabilities present in recent hardware, with FlashAttention-2 achieving only 35% utilization on the H100 GPU. We develop three main techniques to speed up attention on Hopper GPUs: exploiting asynchrony of the Tensor Cores and TMA to (1) overlap overall computation and data movement via warp-specialization and (2) interleave block-wise matmul and softmax operations, and (3) block quantization and incoherent processing that leverages hardware support for FP8 low-precision. We demonstrate that our method, FlashAttention-3, achieves speedup on H100 GPUs by 1.5-2.0× with FP16 reaching up to 740 TFLOPs/s (75% utilization), and with FP8 reaching close to 1.2 PFLOPs/s. We validate that FP8 FlashAttention-3 achieves 2.6× lower numerical error than a baseline FP8 attention.

## 1 Introduction

For the Transformer architecture [59], the attention mechanism constitutes the primary computational bottleneck, since computing the self-attention scores of queries and keys has quadratic scaling in the sequence length. Scaling attention to longer context will unlock new capabilities (modeling and reasoning over multiple long documents [24, 43, 50] and files in large codebases [30, 48]), new modalities (high-resolution images [11], audio [23], video [25]), and new applications (user interaction with long history [53], agent workflow with long horizon [62]). This has generated significant interest in making attention faster in the long-context regime, including by approximation [14, 27, 56] and software optimization ([17, 29, 45]), or even alternative architectures [22, 42, 55].

In this work, we build on the work of Dao et al. [17] on developing exact-attention algorithms that integrate knowledge of the GPU's execution model and hardware characteristics into their high-level design. In [17], Dao et al. introduced FlashAttention, a novel tiling strategy for parallelizing attention that eliminates intermediate reads/writes to slow global memory through fusing all of the attention operations into a single GPU kernel. Dao [15] restructured the algorithm as FlashAttention-2 to also parallelize over the sequence length dimension and perform the inner loop of the forward pass over blocks of the key and value matrices, thus improving the occupancy and distribution of work on the GPU. However, we observe that FlashAttention-2 nonetheless achieves poor utilization on newer GPUs relative to optimized matrix-multiplication (GEMM) kernels, such as 35% vs. 80-90% on the Hopper H100 GPU. Partially, this may be attributed to implementation-level differences, such as not using Hopper-specific instructions in place of Ampere ones when targeting the Tensor Cores. Several work such as ThunkerKitten [52] and cuDNN 9 [39] has shown that with Hopper-specific instructions and tile-based abstractions, one can speedup attention computation and simplify the implementation.

*Equal contribution



arXiv:2407.08608v2 [cs.LG] 12 Jul 2024

# FlashAttention-3: 基于异步处理与低精度技术的快速精准注意力机制

Jay Shah*[1], Ganesh Bikshandi*[1], Ying Zhang[2], Vijay Thakkar[3,4], Pradeep Ramani[3], and Tri Dao[5,6]

[1]Colfax Research [2]Meta [3]NVIDIA [4]Georgia Tech [5]Princeton University [6]Together AI

{jayhshah,ganesh}@colfax-intl.com, yingz@meta.com, {vithakkar,prraman}@nvidia.com, tri@tridao.me

2024年7月16日

**摘要**

Attention, as a core layer of the ubiquitous Transformer architecture, is the bottleneck for large language models and long-context applications. FlashAttention elaborated an approach to speed up attention on GPUs through minimizing memory reads/writes. However, it has yet to take advantage of new capabilities present in recent hardware, with FlashAttention-2 achieving only 35% utilization on the H100 GPU. We develop three main techniques to speed up attention on Hopper GPUs: exploiting asynchrony of the Tensor Cores and TMA to (1) overlap overall computation and data movement via warp-specialization and (2) interleave block-wise matmul and softmax operations, and (3) block quantization and incoherent processing that leverages hardware support for FP8 low-precision. We demonstrate that our method, FlashAttention-3, achieves speedup on H100 GPUs by 1.5-2.0× with FP16 reaching up to 740 TFLOPs/s (75% utilization), and with FP8 reaching close to 1.2 PFLOPs/s. We validate that FP8 FlashAttention-3 achieves 2.6× lower numerical error than a baseline FP8 attention.

## 1 引言

在Transformer架构 [59], 中，注意力机制构成了主要的计算瓶颈，因为计算查询键对的自注意力得分时，其复杂度会随着序列长度的增加而呈二次方增长。若能使注意力机制适配更长的上下文，将能够实现新的功能（对多个长文档 [24, 43, 50] 以及大型代码库中的文件进行建模与推理 [30, 48]）、处理新的数据模态（高分辨率图像 [11], 、音频 [23], 、视频 [25]），并应用于新的场景（具有长历史记录的用户交互 [53], 、具有长时间视界的智能体工作流 [62]）。这一需求催生了人们极大的兴趣，致力于通过近似方法 [14, 27, 56]、软件优化[17, 29, 45]，甚至是采用替代性架构 [22, 42, 55]，来提升长上下文场景下的注意力计算速度。

In this work, we build on the work of Dao et al. [17] on developing exact-attention algorithms that integrate knowledge of the GPU's execution model and hardware characteristics into their high-level design. In [17], Dao et al. introduced FlashAttention, a novel tiling strategy for parallelizing attention that eliminates intermediate reads/writes to slow global memory through fusing all of the attention operations into a single GPU kernel. Dao [15] restructured the algorithm as FlashAttention-2 to also parallelize over the sequence length dimension and perform the inner loop of the forward pass over blocks of the key and value matrices, thus improving the occupancy and distribution of work on the GPU. However, we observe that FlashAttention-2 nonetheless achieves poor utilization on newer GPUs relative to optimized matrix-multiplication (GEMM) kernels, such as 35% vs. 80-90% on the Hopper H100 GPU. Partially, this may be attributed to implementation-level differences, such as not using Hopper-specific instructions in place of Ampere ones when targeting the Tensor Cores. Several work such as ThunkerKitten [52] and cuDNN 9 [39] has shown that with Hopper-specific instructions and tile-based abstractions, one can speedup attention computation and simplify the implementation.

*等量贡献

More fundamentally, FlashAttention-2's algorithm adheres to a simplified synchronous model and makes no explicit use of asynchrony and low-precision in its design. Asynchrony is a result of hardware specialization to accelerate the most important operations in a ML workload: specific hardware units performing matrix multiplication (Tensor Cores) or memory loading (Tensor Memory Accelerator – TMA), separate from the rest of the CUDA cores performing logic, integer, and floating point computation. Low precision such as FP8 in Hopper and FP4 in Blackwell, continuing the trend of FP16 (Pascal in 2017) and BF16 (Ampere in 2020), is a proven technique to get double or quadruple throughput for the same power and chip area. We review the capabilities afforded by Hopper in these directions in §2.2. The technical challenge is to redesign FlashAttention-2 to make use of these hardware features: asynchrony requires overlapping computation between matmul and softmax even though one depends on the output of the other, and low-precision requires care to minimize quantization error, especially in the case of outlier features in LLMs [20, 54].

To this end, we propose FlashAttention-3, which contributes and synthesizes three new ideas to further improve performance on newer GPU architectures:[1]

1. **Producer-Consumer asynchrony:** We define a warp-specialized software pipelining scheme that exploits the asynchronous execution of data movement and Tensor Cores by splitting producers and consumers of data into separate warps, thereby extending the algorithm's ability to hide memory and instruction issue latencies.

2. **Hiding softmax under asynchronous block-wise GEMMs:** We overlap the comparatively low-throughput non-GEMM operations involved in softmax, such as floating point multiply-add and exponential, with the asynchronous WGMMA instructions for GEMM. As part of this, we rework the FlashAttention-2 algorithm to circumvent certain sequential dependencies between softmax and the GEMMs. For example, in the 2-stage version of our algorithm, while softmax executes on one block of the scores matrix, WGMMA executes in the asynchronous proxy to compute the next block.

3. **Hardware-accelerated low-precision GEMM:** We adapt the forward pass algorithm to allow for targeting the FP8 Tensor Cores for GEMM, nearly doubling the measured TFLOPs/s. This requires bridging the different layout conformance requirements of WGMMA in terms of how blocks of FP32 accumulator and FP8 operand matrices are assumed to be laid out in memory. We use the techniques of block quantization and incoherent processing to mitigate the loss of accuracy that results from moving to FP8 precision.

To validate our method empirically, we benchmark FlashAttention-3 on the H100 SXM5 GPU over a range of parameters and show that (1) FP16 achieves 1.5-2.0× speedup over FlashAttention-2 in the forward pass (reaching up to 740 TFLOPs/s) and 1.5-1.75× in the backward pass, (2) FP8 achieves close to 1.2 PFLOPs/s, and (3) for large sequence length, FP16 outperforms and FP8 is competitive[2] with a state-of-the-art implementation of attention from NVIDIA's cuDNN library. We also validate that FP16 FlashAttention-3 yields the same numerical error as FlashAttention-2 and is better than the standard attention implementation as intermediate results (e.g., softmax rescaling) are kept in FP32. Moreover, FP8 FlashAttention-3 with block quantization and incoherent processing is 2.6× more accurate than standard attention with per-tensor quantization in cases with outlier features.

We open-source FlashAttention-3 with a permissive license[3] and plan to integrate it with PyTorch and Hugging Face libraries to benefit the largest number of researchers and developers.

## 2 Background: Multi-Head Attention and GPU Characteristics

### 2.1 Multi-Head Attention

Let $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ be the query, key and value input sequences associated to a single head, where $N$ is the sequence length and $d$ is the head dimension. Then the attention output $\mathbf{O}$ is computed as:

$$\mathbf{S} = \alpha \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

[1]We describe our results in the context of NVIDIA's Hopper architecture. However, our algorithm is operative for any GPU architecture with sufficiently robust asynchronous execution and low-precision capabilities.

[2]More precisely, for head dimension 64 FlashAttention-3 FP8 is ahead, while for head dimensions 128 and 256 it is at par for those cases without causal masking and behind with causal masking.

[3]FlashAttention-3 is available at `https://github.com/Dao-AILab/flash-attention`



More fundamentally, FlashAttention-2's algorithm adheres to a simplified synchronous model and makes no explicit use of asynchrony and low-precision in its design. Asynchrony is a result of hardware specialization to accelerate the most important operations in a ML workload: specific hardware units performing matrix multiplication (Tensor Cores) or memory loading (Tensor Memory Accelerator – TMA), separate from the rest of the CUDA cores performing logic, integer, and floating point computation. Low precision such as FP8 in Hopper and FP4 in Blackwell, continuing the trend of FP16 (Pascal in 2017) and BF16 (Ampere in 2020), is a proven technique to get double or quadruple throughput for the same power and chip area. We review the capabilities afforded by Hopper in these directions in §2.2. The technical challenge is to redesign FlashAttention-2 to make use of these hardware features: asynchrony requires overlapping computation between matmul and softmax even though one depends on the output of the other, and low-precision requires care to minimize quantization error, especially in the case of outlier features in LLMs [20, 54].

为此，我们提出了FlashAttention-3算法，该算法融入并整合了三项全新理念，旨在进一步提升在新型GPU架构上的性能：[1]

1. **生产者-消费者模型的异步处理：**我们设计了一种针对warp架构优化的软件流水线方案，该方案通过将数据的产生者和消费者分配到不同的warp中，充分利用数据传输与张量核心的异步执行能力，进而提升算法隐藏内存访问及指令发送延迟的能力。

2. **在异步分块GEMM运算下隐藏softmax函数：**我们将softmax函数中吞吐量相对较低的非GEMM运算，如浮点数乘加运算和指数运算，与用于GEMM的异步WGMMA指令交错执行。为此，我们对FlashAttention-2算法进行了优化，以规避softmax函数与GEMM运算之间的某些顺序依赖关系。例如，在该算法的两阶段版本中，当softmax函数正在处理得分矩阵的一个块时，WGMMA则会在异步代理环境中运行，从而计算下一个块的数据。

3. **硬件加速的低精度GEMM运算：**我们对前向传播算法进行了优化，使其能够针对GEMM运算使用FP8格式的张量核心，从而使测得的TFLOPs/秒数值几乎翻倍。为此，我们需要解决WGMMA在FP32累加器块与FP8操作数矩阵在内存中的布局要求上的差异问题。通过采用分块量化与非相干处理技术，我们有效降低了因切换到FP8精度而带来的精度损失。

To validate our method empirically, we benchmark FlashAttention-3 on the H100 SXM5 GPU over a range of parameters and show that (1) FP16 achieves 1.5-2.0× speedup over FlashAttention-2 in the forward pass (reaching up to 740 TFLOPs/s) and 1.5-1.75× in the backward pass, (2) FP8 achieves close to 1.2 PFLOPs/s, and (3) for large sequence length, FP16 outperforms and FP8 is competitive[2] with a state-of-the-art implementation of attention from NVIDIA's cuDNN library. We also validate that FP16 FlashAttention-3 yields the same numerical error as FlashAttention-2 and is better than the standard attention implementation as intermediate results (e.g., softmax rescaling) are kept in FP32. Moreover, FP8 FlashAttention-3 with block quantization and incoherent processing is 2.6× more accurate than standard attention with per-tensor quantization in cases with outlier features.

我们以宽松的许可协议[3] 将FlashAttention-3开源，并计划将其与PyTorch及Hugging Face库整合，以便为最多的研究人员和开发者带来益处。

## 2 背景：多头注意力机制与GPU的特性

### 2.1 多头注意力机制

设$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V} \in \mathbb{R}^{N \times d}$分别为对应单个注意力头的查询序列、键序列和值序列，其中 $N$ 表示序列长度， $d$ 表示注意力头维度。此时，注意力输出$\mathbf{O}$ 的计算方式如下：

$$\mathbf{S} = \alpha \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

[1]我们在NVIDIA的Hopper架构背景下展示了我们的实验结果。不过，只要具备足够强大的异步执行能力及低精度运算功能，任何GPU架构均可运行我们的算法。[2]更具体地说，在注意力头维度为64的情况下，FlashAttention-3的FP8格式表现更为优异；而在头部维度为128和256时，若不使用因果掩码则两者性能相当，但一旦启用因果掩码机制，FlashAttention-3的性能就会落后。[3]FlashAttention-3的代码可在https://github.com/Dao-AILab/flash-attention地址获取。

where softmax is applied row-wise and one typically sets $\alpha = 1/\sqrt{d}$ as the scaling factor. In practice, we subtract rowmax($\mathbf{S}$) from $\mathbf{S}$ to prevent numerical instability with the exponential function. For multi-head attention (MHA), each head has its own set of query, key and value projections, and this computation parallelizes across multiple heads and batches to produce the full output tensor.

Now let $\phi$ be a scalar loss function and let $\mathbf{d}(-) = \partial\phi/\partial(-)$ be notation for the gradient. Given the output gradient $\mathbf{dO} \in \mathbb{R}^{N\times d}$, we compute $\mathbf{dQ}$, $\mathbf{dK}$, and $\mathbf{dV}$ according to the chain rule as follows:

$$\begin{aligned}
\mathbf{dV} &= \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N\times d} \\
\mathbf{dP} &= \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N\times N} \\
\mathbf{dS} &= \mathrm{dsoftmax}(\mathbf{dP}) \in \mathbb{R}^{N\times N} \\
\mathbf{dQ} &= \alpha \mathbf{dS}\mathbf{K} \in \mathbb{R}^{N\times d} \\
\mathbf{dK} &= \alpha \mathbf{dS}^\top \mathbf{Q} \in \mathbb{R}^{N\times d},
\end{aligned}$$

Here, we have that $\mathbf{d}s = (\mathrm{diag}(p) - pp^\top)\mathbf{d}p$ for $p = \mathrm{softmax}(s)$ as a function of a vector $s$, and we write dsoftmax($\mathbf{dP}$) for this formula applied row-wise. Finally, this computation again parallelizes across the number of heads and batches for the backward pass of MHA.

## 2.2 GPU hardware characteristics and execution model

We describe the aspects of the GPU's execution model relevant for FlashAttention-3, with a focus on the NVIDIA Hopper architecture as a concrete instantiation of this model.

**Memory hierarchy:** The GPU's memories are organized as a hierarchy of data locales, with capacity inversely related to bandwidth (Table 1)[4]. Global memory (GMEM), also known as HBM, is the off-chip DRAM accessible to all streaming multiprocessors (SMs). Data from GMEM gets transparently cached into an on-chip L2 cache. Next, each SM contains a small on-chip, programmer-managed highly banked cache called shared memory (SMEM). Lastly, there is the register file within each SM.

**Thread hierarchy:** The GPU's programming model is organized around logical groupings of execution units called threads. From the finest to coarsest level, the thread hierarchy is comprised of threads, warps (32 threads), warpgroups (4 contiguous warps), threadblocks (i.e., cooperative thread arrays or CTAs), threadblock clusters (in Hopper), and grids.

These two hierarchies are closely interlinked. Threads in the same CTA are co-scheduled on the same SM, and CTAs in the same cluster are co-scheduled on the same GPC. SMEM is directly addressable by all threads within a CTA, whereas each thread has at most 256 registers (RMEM) private to itself.

Table 1: Thread-Memory hierarchy for the NVIDIA Hopper H100 SXM5 GPU.

| Hardware Level | Parallel Agent | Data Locale | Capacity @ Bandwidth |
|---|---|---|---|
| Chip | Grid | GMEM | 80 GiB @ 3.35 TB/s |
| GPC | Threadblock Clusters | L2 | 50 MiB @ 12 TB/s |
| SM | Threadblock (CTA) | SMEM | 228 KiB per SM, 31TB/s per GPU |
| Thread | Thread | RMEM | 256 KiB per SM |

**Asynchrony and warp-specialization:** GPUs are throughput processors that rely on concurrency and asynchrony to hide memory and execution latencies. For async memory copy between GMEM and SMEM, Hopper has the Tensor Memory Accelerator (TMA) as a dedicated hardware unit [38, §7.29]. Furthermore, unlike prior architectures such as Ampere, the Tensor Core of Hopper, exposed via the warpgroup-wide WGMMA instruction [40, §9.7.14], is also asynchronous and can source its inputs directly from shared memory.

[4]Luo et al. [34] reports shared memory bandwidth of 128 bytes per clock cycle per SM, and we multiply that by 132 SMs and the boost clock of 1830 MHz.



其中会按行应用softmax函数，通常会将 $\alpha = 1/\sqrt{d}$ 设定为缩放因子。在实际应用中，为了避免指数函数引发数值不稳定问题，人们会从$\mathbf{S}$ 中减去rowmax($\mathbf{S}$)。对于多头注意力机制而言，每个注意力头都拥有独立的查询、键和值投影参数，这类计算会在多个注意力头及批次之间并行执行，最终生成完整的输出张量。

现在令 $\phi$ 为一个标量损失函数，同时让$\mathbf{d}(-) = \partial\phi/\partial(-)$作为梯度的符号表示。已知输出梯度为$\mathbf{dO} \in \mathbb{R}^{N\times d}$，我们根据链式法则按如下方式计算$\mathbf{dQ}$、$\mathbf{dK}$以及$\mathbf{dV}$ 。

$$\begin{aligned}
\mathbf{dV} &= \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N\times d} \\
\mathbf{dP} &= \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N\times N} \\
\mathbf{dS} &= \mathrm{dsoftmax}(\mathbf{dP}) \in \mathbb{R}^{N\times N} \\
\mathbf{dQ} &= \alpha \mathbf{dS}\mathbf{K} \in \mathbb{R}^{N\times d} \\
\mathbf{dK} &= \alpha \mathbf{dS}^\top \mathbf{Q} \in \mathbb{R}^{N\times d},
\end{aligned}$$

在这里，我们将$\mathbf{d}s = (\mathrm{diag}(p) - pp^\top)\mathbf{d}p$视为针对向量 $s$的softmax($s$)函数，同时用dsoftmax($\mathbf{dP}$)来表示对该公式进行逐行应用后的结果。最后，在MHA的后向传播过程中，这一计算过程会再次在头数与批次数量上实现并行处理。

## 2.2 GPU硬件特性与执行模型

我们介绍了与FlashAttention-3相关的GPU执行模型各要素，重点探讨了作为该模型具体实现形式的NVIDIA Hopper架构。

**内存层次结构：**GPU的内存被组织成不同层级的数据存储区域，其容量与带宽呈反比关系（见表1）[4]。全局内存（GMEM），亦即高带宽内存，是所有流多处理器（SM）均可访问的片外DRAM。来自GMEM的数据会自动被透明地缓存在片内的L2缓存中。接着，每个流多处理器都包含一个由程序员管理的、具有多个存储单元的小型片内缓存，即共享内存（SMEM）。最后，还有每个流多处理器内部的寄存器文件。

**线程层级结构：**GPU的编程模型是围绕被称为线程的执行单元的逻辑分组来构建的。从最细粒度到最粗粒度，线程层级结构依次包括线程、线程组（32个线程）、线程组串（4个连续的线程组）、线程块（即协作式线程数组或CTA）、Hopper架构中的线程块集群，以及网格结构。

这两个层次结构之间存在着紧密的联系。同一CTA中的线程会被共同调度到同一个SM上，而同一集群中的CTA则会被共同调度到同一个GPC上。CTA内的所有线程都可以直接访问SMEM，而每个线程最多只有256个属于自己的私有寄存器（RMEM）。

表1：NVIDIA Hopper H100 SXM5 GPU的线程-内存层次结构。

| 硬件层面 | 并行智能体 | 数据区域设置 | 带宽条件下的容量 |
|---|---|---|---|
| Chip | Grid | GMEM | 80 GiB，传输速率为3.35 TB/秒的传输速率 |
| GPC | 线程块集群 | L2 | 50MiB，传输速率为12 TB/秒的传输速率 |
| SM | 线程块（控制单元） | SMEM | 每个流多处理器为228 KiB，每颗GPU的传输速率为31 TB/秒的传输速率 |
| 线程 | 线程 | RMEM | 每个流多处理器拥有256 KiB的共享内存 |

**异步处理与指令级专化技术：**GPU属于吞吐量型处理器，它依靠并发性与异步处理来掩盖内存访问和指令执行的延迟。对于在全局内存与流多处理器内存之间进行异步数据复制时，Hopper架构配备了专门的硬件单元——张量内存加速器（TMA） [38，§7.29]。此外，与Ampere等早期架构不同，Hopper架构中的张量核心通过覆盖整个线程组的WGMMA指令 [40, § 9.7.14], 对外暴露，该核心同样支持异步处理，并且能够直接从共享内存中获取输入数据。

[4]Luo等人 [34] 指出每个流多处理器的每个时钟周期对应的共享内存带宽为128字节，我们再将该数值乘以132个流多处理器以及1830 MHz的加速时钟频率。

Hardware support for asynchrony allows for warp-specialized kernels, where the warps of a CTA are divided into producer or consumer roles that only ever issue either data movement or computation. Generically, this improves the compiler's ability to generate optimal instruction schedules [4]. In addition, Hopper supports the dynamic reallocation of registers between warpgroups via `setmaxnreg` [40, §9.7.17.1], so those warps doing MMAs can obtain a larger share of RMEM than those just issuing TMA (for which only a single thread is needed).

**Low-precision number formats:** Modern GPUs have specialized hardware units for accelerating low-precision computation. For example, the WGMMA instruction can target the FP8 Tensor Cores on Hopper to deliver 2x the throughput per SM when compared to FP16 or BF16.

However, correctly invoking FP8 WGMMA entails understanding the layout constraints on its operands. Given a GEMM call to multiply $A \times B^\top$ for an $M \times K$-matrix $A$ and an $N \times K$-matrix $B$, we say that the $A$ or $B$ operand is *mn-major* if it is contiguous in the outer $M$ or $N$ dimension, and *k-major* if is instead contiguous in the inner $K$-dimension. Then for FP16 WGMMA, both mn-major and k-major input operands are accepted for operands in SMEM, but for FP8 WGMMA, only the k-major format is supported. Moreover, in situations such as attention where one wants to fuse back-to-back GEMMs in a single kernel, clashing FP32 accumulator and FP8 operand layouts pose an obstacle to invoking dependent FP8 WGMMAs.

In the context of attention, these layout restrictions entail certain modifications to the design of an FP8 algorithm, which we describe in §3.3.

### 2.3 Standard Attention and Flash Attention

Following Dao et al. [17], we let **standard attention** denote an implementation of attention on the GPU that materializes the intermediate matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM. The main idea of FlashAttention was to leverage a local version of the softmax reduction to avoid these expensive intermediate reads/writes and fuse attention into a single kernel. Local softmax corresponds to lines 18-19 of the consumer mainloop in Algorithm 1 together with the rescalings of blocks of $\mathbf{O}$. The simple derivation that this procedure indeed computes $\mathbf{O}$ can be found in [15, §2.3.1].

## 3 FlashAttention-3: Algorithm

In this section, we describe the FlashAttention-3 algorithm. For simplicity, we focus on the forward pass, with the backward pass algorithm described in Appendix B.1. We first indicate how to integrate warp-specialization with a circular SMEM buffer into the base algorithm of FlashAttention-2. We then explain how to exploit asynchrony of WGMMA to define an overlapped GEMM-softmax 2-stage pipeline. Finally, we describe the modifications needed for FP8, both in terms of layout conformance and accuracy via block quantization and incoherent processing.

### 3.1 Producer-Consumer asynchrony through warp-specialization and pingpong scheduling

**Warp-specialization** As with FlashAttention-2, the forward pass of FlashAttention-3 is embarrassingly parallel in the batch size, number of heads, and query sequence length. Thus, it will suffice to give a CTA-level view of the algorithm, which operates on a tile $\mathbf{Q}_i$ of the query matrix to compute the corresponding tile $\mathbf{O}_i$ of the output. To simplify the description, we first give the warp-specialization scheme with a circular SMEM buffer that does *not* have in addition the GEMM-softmax overlapping. Let $d$ be the head dimension, $N$ the sequence length, and fix a query block size $B_r$ to divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, .., \mathbf{Q}_{T_r}$.



对异步处理的硬件支持使得基于线程组的内核成为可能，这类内核会将CTA中的各个线程组划分为生产者或消费者角色，这些角色仅负责执行数据传输或计算操作。总体而言，这提升了编译器生成最优指令调度计划的能力 [4]。此外，Hopper架构还通过setmaxnreg [40, §9.7.17.1]，功能支持在各个线程组之间动态重新分配寄存器，这样一来，那些执行矩阵乘法操作的线程组就能比仅执行TMA操作的线程组获得更多随机访问内存资源（因为后者仅需一个线程即可）。

**低精度数值格式：**现代GPU配备了专门的硬件单元，用于加速低精度计算。例如，WGMMA指令可以利用Hopper架构上的FP8张量核心，在每个性能流多处理器的吞吐量上实现相比FP16或BF16两倍的提升。

However, correctly invoking FP8 WGMMA entails understanding the layout constraints on its operands. Given a GEMM call to multiply $A \times B^\top$ for an $M \times K$-matrix $A$ and an $N \times K$-matrix $B$, we say that the $A$ or $B$ operand is *mn-major* if it is contiguous in the outer $M$ or $N$ dimension, and *k-major* if is instead contiguous in the inner $K$-dimension. Then for FP16 WGMMA, both mn-major and k-major input operands are accepted for operands in SMEM, but for FP8 WGMMA, only the k-major format is supported. Moreover, in situations such as attention where one wants to fuse back-to-back GEMMs in a single kernel, clashing FP32 accumulator and FP8 operand layouts pose an obstacle to invoking dependent FP8 WGMMAs.

在注意力机制的背景下，这些布局限制要求对FP8算法的设计进行一定的修改，我们将在§3.3节中对这些修改进行阐述。

### 2.3 StandardAttention与Flash Attention

沿袭Dao等人的 [17]，研究思路，我们用**标准注意力机制**来指代在GPU上实现的注意力计算方式，该方式会将**S** 和**P** 这些中间矩阵加载到高带宽内存中。FlashAttention的核心思想在于利用局部版本的softmax函数进行简化，从而避免繁琐的中间数据读写操作，并将注意力计算整合为单个内核。这种局部softmax实现方式对应于算法1中消费者主循环的第18-19行，同时还涉及对**O**对应数据块的重新缩放处理。关于这一方法确实能够计算出**O**的简单推导过程，可见于 [15, §2.3.1]。

## 3 FlashAttention-3: 算法

在本节中，我们将详细介绍FlashAttention-3算法。为简化阐述，我们主要聚焦于前向传播过程，而后向传播的算法则会在附录B.1中说明。首先，我们会介绍如何将指令级专化技术与环形SMEM缓冲区整合到FlashAttention-2的基础算法中。接着，我们会解释如何利用WGMMA的异步处理特性来构建一个重叠式的GEMM-softmax两阶段处理流程。最后，我们将阐述在FP8格式下所需的改进措施，包括布局适配要求，以及通过分块量化和非相干处理技术来提升精度的方法。

### 3.1 生产者-消费者模型中的异步处理、指令级专化技术以及乒乓调度机制

**Warp-specialization** As with FlashAttention-2, the forward pass of FlashAttention-3 is embarrassingly parallel in the batch size, number of heads, and query sequence length. Thus, it will suffice to give a CTA-level view of the algorithm, which operates on a tile $\mathbf{Q}_i$ of the query matrix to compute the corresponding tile $\mathbf{O}_i$ of the output. To simplify the description, we first give the warp-specialization scheme with a circular SMEM buffer that does *not* have in addition the GEMM-softmax overlapping. Let $d$ be the head dimension, $N$ the sequence length, and fix a query block size $B_r$ to divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, .., \mathbf{Q}_{T_r}$.

**Algorithm 1** FlashAttention-3 forward pass **without** intra-consumer overlapping – CTA view

**Require:** Matrices $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, key block size $B_c$ with $T_c = \lceil \frac{N}{B_c} \rceil$.

Initialize pipeline object to manage barrier synchronization with $s$-stage circular SMEM buffer.
**if** in producer warpgroup **then**
  Deallocate predetermined number of registers.
  Issue load $\mathbf{Q}_i$ from HBM to shared memory.
  Upon completion, commit to notify consumer of the load of $\mathbf{Q}_i$.
  **for** $0 \le j < T_c$ **do**
    Wait for the $(j \% s)$th stage of the buffer to be consumed.
    Issue loads of $\mathbf{K}_j, \mathbf{V}_j$ from HBM to shared memory at the $(j \% s)$th stage of the buffer.
    Upon completion, commit to notify consumers of the loads of $\mathbf{K}_j, \mathbf{V}_j$.
  **end for**
**else**
  Reallocate predetermined number of registers as function of number of consumer warps.
  On-chip, initialize $\mathbf{O}_i = (0) \in \mathbb{R}^{B_r \times d}$ and $\ell_i, m_i = (0), (-\infty) \in \mathbb{R}^{B_r}$.
  Wait for $\mathbf{Q}_i$ to be loaded in shared memory.
  **for** $0 \le j < T_c$ **do**
    Wait for $\mathbf{K}_j$ to be loaded in shared memory.
    Compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T$ (SS-GEMM). Commit and wait.
    Store $m_i^{\text{old}} = m_i$ and compute $m_i = \max(m_i^{\text{old}}, \text{rowmax}(\mathbf{S}_i^{(j)}))$.
    Compute $\widetilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i)$ and $\ell_i = \exp(m_i^{\text{old}} - m_i)\ell_i + \text{rowsum}(\widetilde{\mathbf{P}}_i^{(j)})$.
    Wait for $\mathbf{V}_j$ to be loaded in shared memory.
    Compute $\mathbf{O}_i = \text{diag}(\exp(m_i^{\text{old}} - m_i))^{-1}\mathbf{O}_i + \widetilde{\mathbf{P}}_i^{(j)}\mathbf{V}_j$ (RS-GEMM). Commit and wait.
    Release the $(j \% s)$th stage of the buffer for the producer.
  **end for**
  Compute $\mathbf{O}_i = \text{diag}(\ell_i)^{-1}\mathbf{O}_i$ and $L_i = m_i + \log(\ell_i)$.
  Write $\mathbf{O}_i$ and $L_i$ to HBM as the $i$th block of $\mathbf{O}$ and $L$.
**end if**

For our implementation of Algorithm 1 on Hopper, we use `setmaxnreg` for (de)allocations, TMA for loads of $\mathbf{Q}_i$ and $\{\mathbf{K}_j, \mathbf{V}_j\}_{0 \le j < T_c}$, and WGMMA to execute the GEMMs in the consumer mainloop, with the SS or RS prefix indicating whether the first operand is sourced from shared memory or register file. For interpreting the execution flow of Algorithm 1, note that issuing TMA loads does not stall on the completion of other loads due to asynchrony. Moreover, in the producer mainloop, no waits will be issued for the first $s$ iterations as the buffer gets filled.

**Pingpong scheduling** The asynchronous nature of WGMMA and TMA, along with warp-specialization, opens up the opportunity to overlap the softmax computation of one warpgroup with the GEMM of another warpgroup. To motivate this, notice that non-matmul operations have much lower throughput than matmul operations on modern hardware accelerators. As an example, the H100 SXM5 GPU has 989 TFLOPS of FP16 matmul but only 3.9 TFLOPS of special functions such as exponential[5] (necessary for softmax). For the attention forward pass in FP16 with head dimension 128, there are 512x more matmul FLOPS compared to exponential operations, but the exponential has 256x lower throughput, so exponential can take 50% of the cycle compared to matmul. The situation is even worse with FP8, where the matmul throughput doubles but the exponential throughput stays the same.

Since the exponential is performed by a separate hardware unit (the multi-function unit), ideally we'd want the exponential calculation to be scheduled when the Tensor Cores are performing the matmul. To do so, we use synchronization barriers (`bar.sync` instructions) to force the GEMMs (GEMM1 – **PV** of one iteration, and GEMM0 – $\mathbf{QK}^\top$ of the next iteration) of warpgroup 1 to be scheduled before the GEMMs of warpgroup 2. As a result, the softmax of warpgroup 1 will be scheduled while warpgroup 2 is performing its GEMMs. Then the roles swap, with warpgroup 2 doing softmax while warpgroup 1 doing GEMMs (hence, "pingpong" scheduling). This is illustrated

[5]The CUDA programming guide specifies that 16 operations of special functions can be performed per streaming multiprocessor (SM) per clock cycle. We multiply 16 by 132 SMs and 1830 MHz clock speed to get 3.9 TFLOPS of special functions.



**算法1**不含消费者内部重叠现象的FlashAttention-3前向传播——CTA视角

**所需条件：** HBM中的$\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$矩阵与$\mathbf{K}$、$\mathbf{V} \in \mathbb{R}^{N \times d}$矩阵，以及大小为 $B_c$、具有 $T_c = \lceil \frac{N}{B_c} \rceil$特性的键块大小。

初始化管道对象，用于通过与 $s$阶段循环式SMEM缓冲区实现屏障同步机制。
**如果位于**生产者线程组中**则**
  释放预定数量的寄存器。
  将来自高带宽内存的$\mathbf{Q}_i$ 数据执行数据加载操作，传输至共享内存中。
  操作完成后，需立即通知各消费者有关$\mathbf{Q}_i$的加载情况。
  **for** $0 \le j < T_c$ **do**
    等待缓冲区中的第$(j \% s)$个阶段被处理完毕。
    在缓冲区的第$(j \% s)$个阶段，将$\mathbf{K}_j$、$\mathbf{V}_j$这些数据从高带宽内存加载到共享内存中。
    操作完成后，同样需要通知各消费者$\mathbf{K}_j$、$\mathbf{V}_j$的加载结果。
  **end for**
**else**
  根据消费者线程组的数量，重新分配预定数量的寄存器。
  在芯片内部，初始化<span style='id':'1'>O$_i = (0) \in \mathbb{R}^{B_r \times d}$以及<span style='id':'5'>l$_i$<span style='id':'8'>，$m_i = (0)$<span style='id':'11'>，$(-\infty) \in \mathbb{R}^{B_r}$。
  等待<span style='id':'1'>Q$_i$ 被加载到共享内存中。
  **for** $0 \le j < T_c$ **do**
    等待<span style='id':'1'>K$_j$ 被加载到共享内存中。
    计算$\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T$ （SS-GEMM运算）。提交后等待。
    存储 $m_i^{\text{old}} = m_i$并计算 $m_i = \max(m_i^{\text{old}},$ 行最大值$\mathbf{S}_i^{(j)})$。
    计算 $\widetilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i)$以及$l_i = \exp(m_i^{\text{old}} - m_i)l_i + \text{rowsum}(\widetilde{\mathbf{P}}^{(j)}{}_i)$。
    请等待$\mathbf{V}_j$ 被加载到共享内存中。
    计算 $\mathbf{O}_i = \text{diag}(\exp(m_i^{\text{old}} - m_i))^{-1}\mathbf{O}_i + \widetilde{\mathbf{P}}^{(j)}{}_i \mathbf{V}_j$(RS-GEMM)。提交后等待。
    为生产者释放缓冲区的第$(j \% s)$阶段空间。
  **结束循环**
  计算 $\mathbf{O}_i = \text{diag}(l_i)^{-1}\mathbf{O}_i$ 以及 $L_i = m_i + \log(l_i)$。
  将$\mathbf{O}_i$ 和 $L_i$ 写入高带宽内存，作为$\mathbf{O}$ 与 $L$的第 $i$个数据块。
**结束if语句**

For our implementation of Algorithm 1 on Hopper, we use `setmaxnreg` for (de)allocations, TMA for loads of $\mathbf{Q}_i$ and $\{\mathbf{K}_j, \mathbf{V}_j\}_{0 \le j < T_c}$, and WGMMA to execute the GEMMs in the consumer mainloop, with the SS or RS prefix indicating whether the first operand is sourced from shared memory or register file. For interpreting the execution flow of Algorithm 1, note that issuing TMA loads does not stall on the completion of other loads due to asynchrony. Moreover, in the producer mainloop, no waits will be issued for the first $s$ iterations as the buffer gets filled.

**Pingpong scheduling** The asynchronous nature of WGMMA and TMA, along with warp-specialization, opens up the opportunity to overlap the softmax computation of one warpgroup with the GEMM of another warpgroup. To motivate this, notice that non-matmul operations have much lower throughput than matmul operations on modern hardware accelerators. As an example, the H100 SXM5 GPU has 989 TFLOPS of FP16 matmul but only 3.9 TFLOPS of special functions such as exponential[5] (necessary for softmax). For the attention forward pass in FP16 with head dimension 128, there are 512x more matmul FLOPS compared to exponential operations, but the exponential has 256x lower throughput, so exponential can take 50% of the cycle compared to matmul. The situation is even worse with FP8, where the matmul throughput doubles but the exponential throughput stays the same.

由于指数运算是由独立的硬件单元（多功能单元）来执行的，理想情况下我们希望在张量核心执行矩阵乘法时同步进行指数计算。为此，我们使用同步屏障（bar.sync指令），强制让线程组1中的GEMM运算（即某次迭代中的GEMM1 – **PV** 以及下一次迭代中的GEMM0 – $\mathbf{QK}^\top$ ）先于线程组2的GEMM运算被调度执行。这样一来，当线程组2正在执行其GEMM运算时，线程组1的softmax运算就会被安排执行。随后两者角色互换，由线程组2负责执行softmax运算，而线程组1则负责执行GEMM运算（因此称为“乒乓式”调度）。示意图如下所示：

[5]根据CUDA编程指南的规定，每个时钟周期内每个流多处理器（SM）可执行16次特殊函数运算。将16乘以132个SM以及1830 MHz的时钟频率，即可得出该系统在特殊函数运算方面的性能为3.9 TFLOPs数值。

in Fig. 1. Though in practice the pingpong scheduling is not as clean as depicted in the figure, we generally find this to improve performance (e.g., from 570 TFLOPS to 620-640 TFLOPS for FP16 forward with head dimension 128 and sequence length 8192).

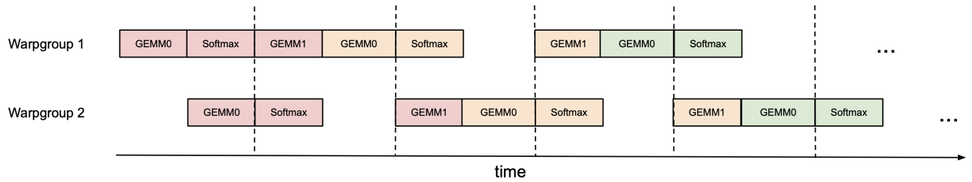



Figure 1: Pingpong scheduling for 2 warpgroups to overlap softmax and GEMMs: the softmax of one warpgroup should be scheduled when the GEMMs of another warpgroup are running. The same color denotes the same iteration.

**Attention variants** For multi-query attention [51] and grouped query attention [3], we follow the approach in FlashAttention-2 and adjust the tensor indexing to avoid duplicating $\mathbf{K}$ and $\mathbf{V}$ in HBM.

### 3.2 Intra-warpgroup overlapping GEMMs and softmax

Even within one warpgroup, we can overlap some instructions in the softmax with some instructions in the GEMMs. We describe one technique to do so.

In the attention algorithm, operations within the inner loop (main loop) have sequential dependencies that impede parallelization within a single iteration. For example, (local) softmax (lines 18 to 19) relies on the output $\mathbf{S}_i^{(j)}$ of the first GEMM, while the second GEMM takes its result $\widetilde{\mathbf{P}}_i^{(j)}$ as an operand. Indeed, the wait statements in lines 17 and 21 of Algorithm 1 serialize the execution of softmax and GEMMs. However, we can break these dependencies by pipelining across iterations through additional buffers in registers. Pursuing this idea, we propose the following two-stage[6] GEMM-softmax pipelining algorithm:

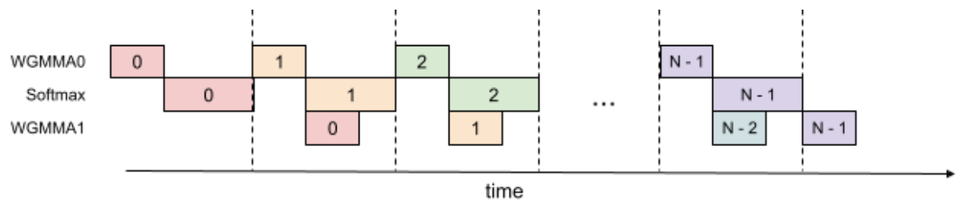



Figure 2: 2-stage WGMMA-softmax pipelining

[6]Note that the number of stages of the overlapping scheme is bounded by, but need not equal, the number $s$ of stages in the circular SMEM buffer.



在图1中。尽管在实际应用中，乒乓调度并不像图中所示的那样理想，但我们通常发现它能提升性能（例如，对于头部维度为128、序列长度为8192的FP16格式前向传播，性能可从570 TFLOPs提升至620-640 TFLOPs）。

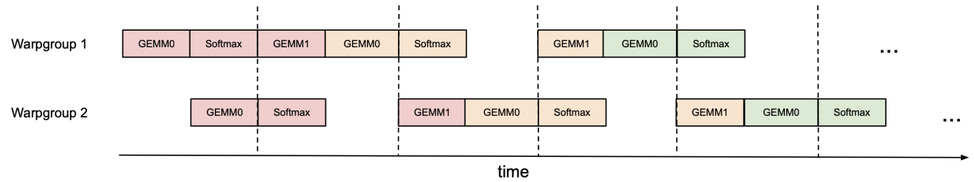



图1：两个线程组采用乒乓调度来同时执行softmax函数与GEMM运算——当一个线程组的GEMM运算正在运行时，另一个线程组的softmax函数则被安排执行。相同颜色表示同一轮迭代。

**注意力机制的变体**对于多查询注意力机制 [51] 和分组查询注意力机制 [3]，，我们采用了FlashAttention-2中的方法，并对张量索引进行了调整，以避免在高带宽内存中重复存储**键向量**和**值向量**。

### 3.2 线程组内的重叠GEMM运算与softmax函数

即便在同一个线程组内，我们也可以将softmax运算中的部分指令与GEMM运算中的部分指令进行重叠处理。本文将介绍一种实现这一目标的技术。

在注意力算法中，内层循环（主循环）中的各项操作存在顺序依赖关系，这阻碍了单次迭代内的并行化处理。例如，(局部)softmax运算（第18至19行）依赖于第一次GEMM的$\mathbf{S}_i^{(j)}$输出，而第二次GEMM则将第一次运算的结果 $\widetilde{\mathbf{P}}^{(j)}{}_i$作为操作数。实际上，算法1中第17行和第21行的等待语句使得softmax运算与GEMM运算的执行变得串行化。不过，我们可以通过在寄存器中利用额外的缓冲区来实现跨迭代的流水线处理，从而打破这些依赖关系。基于这一思路，我们提出了如下两阶段的[6] GEMM-softmax流水线技术：

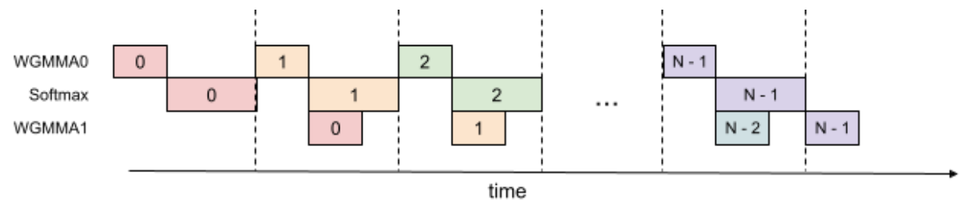



图2：两阶段WGMMA-softmax流水线算法

[6]请注意，重叠方案的阶段数受到圆形SMEM缓冲区中 $s$ 阶段数的限制，但不一定与之相等。

**Algorithm 2** FLASHATTENTION-3 consumer warpgroup forward pass

**Require:** Matrices $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, key block size $B_c$ with $T_c = \lceil \frac{N}{B_c} \rceil$.
1: Reallocate predetermined number of registers as function of number of consumer warps.
2: On-chip, initialize $\mathbf{O}_i = (0) \in \mathbb{R}^{B_r \times d}$ and $\ell_i, m_i = (0), (-\infty) \in \mathbb{R}^{B_r}$.
3: Wait for $\mathbf{Q}_i$ and $\mathbf{K}_0$ to be loaded in shared memory.
4: Compute $\mathbf{S}_{\text{cur}} = \mathbf{Q}_i \mathbf{K}_0^T$ using WGMMA. Commit and wait.
5: Release the 0th stage of the buffer for $\mathbf{K}$.
6: Compute $m_i$, $\tilde{\mathbf{P}}_{\text{cur}}$ and $\ell_i$ based on $\mathbf{S}_{\text{cur}}$, and rescale $\mathbf{O}_i$.
7: **for** $1 \le j < T_c - 1$ **do**
8: Wait for $\mathbf{K}_j$ to be loaded in shared memory.
9: Compute $\mathbf{S}_{\text{next}} = \mathbf{Q}_i \mathbf{K}_j^T$ using WGMMA. Commit but do not wait.
10: Wait for $\mathbf{V}_{j-1}$ to be loaded in shared memory.
11: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_{\text{cur}} \mathbf{V}_{j-1}$ using WGMMA. Commit but do not wait.
12: Wait for the WGMMA $\mathbf{Q}_i \mathbf{K}_j^T$.
13: Compute $m_i$, $\tilde{\mathbf{P}}_{\text{next}}$ and $\ell_i$ based on $\mathbf{S}_{\text{next}}$.
14: Wait for the WGMMA $\tilde{\mathbf{P}}_{\text{cur}} \mathbf{V}_{j-1}$ and then rescale $\mathbf{O}_i$
15: Release the $(j \,\%\, s)$th, resp. $(j - 1 \,\%\, s)$th stage of the buffer for $\mathbf{K}$, resp. $\mathbf{V}$.
16: Copy $\mathbf{S}_{\text{next}}$ to $\mathbf{S}_{\text{cur}}$.
17: **end for**
18: Wait for $\mathbf{V}_{T_c - 1}$ to be loaded in shared memory.
19: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_{\text{last}} \mathbf{V}_{T_c - 1}$ using WGMMA. Commit and wait.
20: Epilogue: Rescale $\mathbf{O}_i$ based on $m_i$. Compute $L_i$ based on $m_i$ and $\ell_i$. Write $\mathbf{O}_i$ and $L_i$ to HBM as the $i$-th block of $\mathbf{O}$ and $L$.

Algorithm 2 functions as a replacement for the consumer path of Algorithm 1 to comprise the complete FLASHATTENTION-3 algorithm for FP16 precision. At a high-level, we use WGMMA as a metonym for asynchronous GEMM. Within the mainloop (lines 8 to 16), the second WGMMA operation of iteration $j$ (line 11) is overlapped with softmax operations from iteration $j + 1$ (line 13).

While the pipelined structure illustrated above offers theoretical performance gains, there are several practical aspects to consider:

**Compiler reordering** The pseudocode represents an idealized execution order but the compiler (NVCC) often rearranges instructions for optimization. This can disrupt the carefully crafted WGMMA and non-WGMMA operation pipelining sequence, potentially leading to unexpected behavior or diminished performance gains. An analysis of the SASS code shows that the compiler generates overlapped code as expected (Section B.2).

**Register pressure** To maintain optimal performance, register spilling should be minimized. However, the 2-stage pipeline requires additional registers to store intermediate results and maintain context between stages. Specifically, an extra $\mathbf{S}_{\text{next}}$ must be kept in registers, leading to extra register usage of size $B_r \times B_c \times$sizeof(float) per threadblock. This increased register demand may conflict with using larger block sizes (another common optimization), which is also register-hungry. In practice, trade-offs should be made based on profiling results.

**3-stage pipelining** Extending the 2-stage algorithm described above, we propose a 3-stage variant that would further overlap the second WGMMA with softmax. While this approach offers the potential for even higher Tensor Core utilization, it requires even more registers due to an additional stage in the pipeline, making the trade-off between tile size and pipeline depth more difficult to balance. A detailed description of the 3-stage algorithm and its evaluation results can be found in Appendix B.3.

### 3.3 Low-precision with FP8

**Efficiency: layout transformations.** Computing the forward pass of FLASHATTENTION-3 in FP8 precision poses additional challenges not encountered for FP16 in terms of layout conformance.



**算法2** FlashAttention-3算法中消费者线程组的前向传播

**所需条件：** HBM中的$\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$和$\mathbf{K}$，以及$\mathbf{V} \in \mathbb{R}^{N \times d}$矩阵，键块大小为 $B_c$，并满足 $T_c = \lceil \frac{N}{B_c}$要求$\rceil$.
1: 根据消费者线程组的数量，重新分配预定数量的寄存器。
2: 在芯片内部，初始化 $\mathbf{O}_i = (0) \in \mathbb{R}^{B_r \times d}$ 以及 $l_i$，$m_i = (0)$，$(-\infty) \in \mathbb{R}^{B_r}$。
3: 等待$\mathbf{Q}_i$ 和$\mathbf{K}_0$ 被加载到共享内存中。
4: 使用WGMMA运算计算$\mathbf{S}_{\text{cur}} = \mathbf{Q}_i \mathbf{K}_0^T$，随后提交并等待。
5: 释放用于**键向量**的缓冲区第0阶段数据。
6: 计算 $m_i$，$\tilde{\mathbf{P}}_{\text{cur}}$基于$\mathbf{S}_{\text{cur}}$的$m_i$ 与$\mathbf{S}_{\text{cur}}$的$\ell_i$，并对$\mathbf{O}_i$进行重新缩放。
7: **for** $1 \le j < T_c - 1$ **do**
8: 请等待**键向量**$j$ 被加载到共享内存中。
9: 使用WGMMA运算计算$\mathbf{S}_{\text{next}} = \mathbf{Q}_i \mathbf{K}_j^T$。立即提交，无需等待。
10: 请等待$\mathbf{V}_{j-1}$ 被加载到共享内存中。
11: 计算$\mathbf{O}_i = \mathbf{O}_i +$的值。通过WGMMA运算计算$\tilde{\mathbf{P}}_{\text{cur}} \mathbf{V}_{j-1}$，执行提交操作但不需要等待。
12: 等待WGMMA运算完成的$\mathbf{Q}_i \mathbf{K}_j^T$结果。
13: 计算 $m_i$的值。根据$\mathbf{S}_{\text{next}}$计算$\tilde{\mathbf{P}}_{\text{next}}$ 以及$l_i$。
14: 等待执行WGMMA运算。$\tilde{\mathbf{P}}_{\text{cur}} \mathbf{V}_{j-1}$，随后对$\mathbf{O}$进行缩放处理。
15: 释放缓冲区中第$j \,\%\, s$位的数据，用于存储$\mathbf{K}$；同时释放第$j - 1 \,\%\, s$位的数据，用于存储$\mathbf{V}$。
16: 将$\mathbf{S}_{\text{next}}$ 的值复制到$\mathbf{S}_{\text{cur}}$中。
17: **此处结束**。
18: 等待$\mathbf{V}_{T_c - 1}$ 的值被加载到共享内存中。
19: 计算 $\mathbf{O}$**值**$_i =$**O值**$_i +$ $\tilde{\mathbf{P}}_{\text{last}} \mathbf{V}_{T_c - 1}$ 通过WGMMA运算完成。提交后等待即可。
20: 尾声阶段：根据 $m_i$对$\mathbf{O}_i$ 进行重新缩放。基于 $m_i$ 以及$l_i$计算 $L_i$。将$\mathbf{O}_i$ 和 $L_i$ 作为第 $i$个块写入高带宽内存中。$\mathbf{O}$**值**与 $L$的。

算法2充当算法1中用户端路径的替代方案，从而构成适用于FP16精度的完整FlashAttention-3算法。从宏观层面来看，我们使用WGMMA来指代异步GEMM运算。在主循环中（第8行至第16行），迭代 $j$ 中的第二次WGMMA运算（第11行）会与迭代 $j + 1$ 中的softmax运算（第13行）同时进行。

尽管上文所示的流水线结构在理论上能提升性能，但仍有若干实际因素需要考虑：

**编译器重排**该伪代码展示的是理想的执行顺序，但编译器（NVCC）往往会为了优化而重新排列指令。这可能会打破精心设计的WGMMA与非WGMMA运算流水线顺序，从而导致异常行为或降低性能提升效果。对SASS代码的分析表明，编译器如预期般生成了重叠的代码（见B.2节）。

**寄存器压力**为保持最佳性能，应尽量减少寄存器溢出现象。然而，两阶段处理流程需要额外的寄存器来存储中间结果并维持各阶段之间的上下文信息。具体而言，必须在寄存器中保留额外的$\mathbf{S}_{\text{next}}$，这会导致每个线程块多占用 $B_r \times B_c \times$sizeof(float)大小的寄存器资源。如此增加的寄存器需求可能会与使用更大的块大小这一常见优化手段产生冲突，因为后者同样会大量消耗寄存器。在实际应用中，应根据性能分析结果来进行权衡取舍。

三**阶段流水线处理** 在上述两阶段算法的基础上，我们提出了一个三阶段变体，该变体进一步将第二个WGMMA运算与softmax函数重叠执行。虽然这种方法有望实现更高的Tensor Core利用率，但由于流水线增加了新的处理阶段，因此需要更多的寄存器，这使得在块大小与流水线深度之间进行权衡变得更加困难。关于三阶段算法的详细描述及其评估结果可见附录B.3。

### 3.3 使用FP8格式的低精度计算

**效率：布局转换**。以FP8精度执行FlashAttention-3的向前传播会在布局一致性方面带来FP16所没有的额外挑战。

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| T0 {d0, d1} | T1 {d0, d1} | T2 {d0, d1} | T3 {d0, d1} | T0 {d4, d5} | T1 {d4, d5} | T2 {d4, d5} | T3 {d4, d5} |
| T0 {d2, d3} | T1 {d2, d3} | T2 {d2, d3} | T3 {d2, d3} | T0 {d6, d7} | T1 {d6, d7} | T2 {d6, d7} | T3 {d6, d7} |

Figure 3: FP32 accumulator register WGMMA layout – rows 0 and 8, threads 0-3, entries 0-7.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| T0 {a0, a1} | T0 {a2, a3} | T1 {a0, a1} | T1 {a2, a3} | T2 {a0, a1} | T2 {a2, a3} | T3 {a0, a1} | T3 {a2, a3} |
| T0 {a4, a5} | T0 {a6, a7} | T1 {a4, a5} | T1 {a6, a7} | T2 {a4, a5} | T2 {a6, a7} | T3 {a4, a5} | T3 {a6, a7} |

Figure 4: FP8 operand A register WGMMA layout – rows 0 and 8, threads 0-3, entries 0-7.

First, we note that the input tensors **Q**, **K**, and **V** are typically given as contiguous in the head dimension, while to satisfy the k-major constraint on FP8 WGMMA for the second GEMM we need **V**, or rather the tiles of **V** loaded into SMEM, to be contiguous in the sequence length dimension. Since the TMA load itself cannot change the contiguous dimension, we then need to either (1) transpose **V** in GMEM as a pre-processing step, or (2) do an in-kernel transpose of tiles of **V** after loading them into SMEM. To implement option (1), we can either (1a) fuse the transpose to the epilogue of a preceding step such as the rotary embedding, or (1b) call a standalone pre-processing transpose kernel[7] to exchange the strides of the sequence length and head dimensions. However, (1a) is difficult to integrate into a standard library, and (1b) is too wasteful in a memory-bound situation such as inference.

Instead, for FP8 FlashAttention-3 we opt for option (2). For the in-kernel transpose, we take advantage of the LDSM (`ldmatrix`) and STSM (`stmatrix`) instructions, which involve a warp of threads collectively loading SMEM to RMEM and storing RMEM to SMEM at a granularity of 128 bytes.[8] The LDSM/STSM instructions are both register efficient, allowing us to execute them in the producer warpgroup, and capable of transposing layouts when doing memory copy. Moreover, after the first iteration we can arrange for the transpose of the next **V** tile to be executed in the shadow of the two WGMMAs that involve the preceding **V** and current **K** tile.

Second, we observe that unlike with FP16, the memory layout of the FP32 accumulator of an FP8 WGMMA is different from that assumed for its operand A when held in registers. We depict fragments of these two layouts in Fig. 3 and Fig. 4, where the entries are held in registers per thread in the listed order. By using byte permute instructions, we can then transform the first WGMMA's accumulator into a format suitable for the second WGMMA, and compatibly with the layout of the **V** tile produced by the in-kernel transpose. Specifically, with reference to Fig. 3, we change the order in sequence to

`{d0 d1 d4 d5 d2 d3 d6 d7}`,

and this register permutation is then replicated over every 8 bytes. In terms of the logical shape of the **P** tile, this manuever permutes its columns (e.g., columns 0189 now become the first four columns). For WGMMA to then compute the correct output tile, we can correspondingly arrange for the in-kernel transpose to write out a matching row permutation of the **V** tile.[9]

**Accuracy: block quantization and incoherent processing.** With FP8 (e4m3) format, one only uses 3 bits to store the mantissa and 4 bits for the exponent. This results in higher numerical error than FP16/BF16. Moreover, large models typically have outlier values [20, 54] that are much larger in magnitude than most other values, making quantization difficult. One typically use per-tensor scaling [37] by keeping one scalar per tensor (e.g., one for **Q**, for **K**, and for **V**). To reduce the numerical error of attention in FP8, we employ two techniques:

1. **Block quantization**: we keep one scalar per block, so that for each of **Q**, **K**, **V** we split the tensor into blocks of size $B_r \times d$ or $B_c \times d$ and quantize them separately. This quantization can be fused with an operation right before attention (e.g., rotary embedding) with no additional slow down (since rotary embedding is memory-bandwidth bound). As the FlashAttention-3 algorithm naturally operates on blocks, we can scale each block of **S** to account for this block quantization at no computation cost.

[7] An optimized transpose kernel will achieve speed near the bandwidth of the device [46].

[8] In the PTX documentation, LDSM/STSM are described as copying $8 \times 8$ matrices with 16-bit entries [40, §9.7.13.4.15-16], but we can pack 8-bit entries two at a time to use LDSM/STSM in the context of FP8 precision. However, the transpose versions of LDSM/STSM cannot split packed 8-bit entries, which necessitates certain register movements in between LDSM and STSM to actually perform a tile-wise transpose; we omit the details.

[9] This additional freedom afforded by doing the in-kernel transpose eliminates having to use shuffle instructions to change register ownership across threads, which we previously described in [7].



| | | | | | | | |
|---|---|---|---|---|---|---|---|
| T0 {d0, d1} | T1 {d0, d1} | T0 {d4,d5} | T1 {d4,d5} | T2 {d0, d1} | T3 {d0,d1} | T2 {d4,d5} | T3 {d4,d5} |
| T0 {d2, d3} | T1 {d2, d3} | T0 {d6,d7} | T1 {d6,d7} | T2 {d2, d3} | T3 {d2,d3} | T2 {d6,d7} | T3 {d6,d7} |

Fi图3：FP32格式累加器寄存器的WGMMA布局——第0行和第8行，线程0-3，元素0-7。

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| T0 {a0, a1} | T0 {a2, a3} | T1 {a0, a1} | T1 {a2, a3} | T2 {a0, a1} | T2 {a2, a3} | T3 {a0, a1} | T3 {a2, a3} |
| T0 {a4, a5} | T0 {a6, a7} | T1 {a4, a5} | T1 {a6, a7} | T2 {a4, a5} | T2 {a6, a7} | T3 {a4, a5} | T3 {a6, a7} |

图4：FP8格式操作数A的寄存器WGMMA布局——第0行和第8行，线程0-3，条目0-7。

First, we note that the input tensors **Q**, **K**, and **V** are typically given as contiguous in the head dimension, while to satisfy the k-major constraint on FP8 WGMMA for the second GEMM we need **V**, or rather the tiles of **V** loaded into SMEM, to be contiguous in the sequence length dimension. Since the TMA load itself cannot change the contiguous dimension, we then need to either (1) transpose **V** in GMEM as a pre-processing step, or (2) do an in-kernel transpose of tiles of **V** after loading them into SMEM. To implement option (1), we can either (1a) fuse the transpose to the epilogue of a preceding step such as the rotary embedding, or (1b) call a standalone pre-processing transpose kernel[7] to exchange the strides of the sequence length and head dimensions. However, (1a) is difficult to integrate into a standard library, and (1b) is too wasteful in a memory-bound situation such as inference.

对于FP8 FlashAttention-3，我们选择了选项（2）。在实现内核内转置时，我们利用了LDSM（ldmatrix）和STSM（stmatrix）指令，这些指令能让一个线程组共同以128字节的粒度将SMEM中的数据加载到RMEM中，再将RMEM中的数据存储回SMEM。[8] LDSM/STSM指令在寄存器使用效率方面表现优异，既允许我们在生产者线程组中执行它们，还能在进行内存复制时完成布局的转置。此外，在第一轮迭代之后，我们可以安排对下一个**V** 块进行转置的操作，使其在处理前一个**V** 块和当前**K** 块的相关WGMMA运算的同时同步执行。

其次，我们发现与FP16不同，FP8 WGMMA的FP32累加器的内存布局与其操作数A存储在寄存器中的布局并不一致。图3和图4展示了这两种布局的片段，其中各条目按照所列顺序存储在每个线程的寄存器中。通过使用字节重排指令，我们可以将第一个WGMMA的累加器转换为适合第二个WGMMA的格式，并使其与内核内转置生成的**V** 块布局相兼容。具体而言，参考图3，我们需要依次改变这些元素的存储顺序。

`{d0 d1 d4 d5 d2 d3 d6 d7}`,

随后，这种寄存器重排操作会在每8个字节的数据上重复执行。从**P** 块的结构特征来看，这一操作会对其列进行重排（例如，原本的0、1、8、9列会变成前四列）。为使WGMMA能够计算出正确的输出块，我们可以相应地安排通过内核内转置来生成**V** 块的对应行重排顺序[9]。

**精度：分块量化与非相干处理技术**。采用FP8（e4m3）格式时，仅需3位来存储尾数，4位用于存储指数。这会导致其数值误差比FP16/BF16更大。此外，大型模型通常存在一些 [20, 54] 异常值，其数值幅度远大于其他大多数值，从而增加了量化难度。人们通常会通过为每个张量保留一个标量来实现张量级缩放 [37] （例如，分别为**Q**、**K**和**V**各设置一个标量）。为降低FP8环境下的注意力机制的数值误差，我们采用了两种技术：

1. **分块量化**：我们为每个分块保留一个标量，因此对于**Q**、**K**和**V** ，都会将对应的张量分割成大小为 $B_r \times d$ 或 $B_c \times d$ 的分块并分别进行量化。这种量化方式可以与注意力计算之前的操作（例如旋转嵌入操作）直接结合，而不会带来额外的性能损耗（因为旋转嵌入操作本身受内存带宽限制）。由于FlashAttention-3算法本身就是以分块形式运行的，我们可以无需任何计算开销即可对**S** 的每个分块应用缩放因子，以此来弥补分块量化带来的影响。

[7] An optimized transpose kernel will achieve speed near the bandwidth of the device [46].

[8] In the PTX documentation, LDSM/STSM are described as copying $8 \times 8$ matrices with 16-bit entries [40, §9.7.13.4.15-16], but we can pack 8-bit entries two at a time to use LDSM/STSM in the context of FP8 precision. However, the transpose versions of LDSM/STSM cannot split packed 8-bit entries, which necessitates certain register movements in between LDSM and STSM to actually perform a tile-wise transpose; we omit the details.

[9] This additional freedom afforded by doing the in-kernel transpose eliminates having to use shuffle instructions to change register ownership across threads, which we previously described in [7].

2. **Incoherent processing**: to even out outliers, we multiply $\mathbf{Q}$ and $\mathbf{K}$ with a random orthogonal matrix $\mathbf{M}$ before quantizing to FP8. Since $\mathbf{M}$ is orthogonal, $\mathbf{M}\mathbf{M}^\top = I$ and so $(\mathbf{Q}\mathbf{M})(\mathbf{K}\mathbf{M})^\top = \mathbf{Q}\mathbf{K}^\top$, i.e., multiplying both $\mathbf{Q}$ and $\mathbf{K}$ with $\mathbf{M}$ does not change the attention output. This serves to "spread out" the outliers since each entry of $\mathbf{Q}\mathbf{M}$ or $\mathbf{K}\mathbf{M}$ is a random sum of entries of $\mathbf{Q}$ or $\mathbf{K}$, thus reducing quantization error. In practice, we follow Chee et al. [9] and Tseng et al. [58] and choose $\mathbf{M}$ to be the product of random diagonal matrices of $\pm 1$ and a Hadamard matrix, which can be multiplied in $O(d \log d)$ instead of $O(d^2)$, and can also be fused with the rotary embedding at no extra computation cost.

We validate that these two techniques reduces numerical error by up to 2.6× in §4.3.

# 4 Empirical Validation

We use the primitives from CUTLASS [57] such as WGMMA and TMA abstractions to implement FlashAttention-3 and evaluate its efficiency and accuracy.

- **Benchmarking attention.** We measure the runtime of FlashAttention-3 across different sequence lengths and compare it to a standard implementation in PyTorch, FlashAttention-2, FlashAttention-2 in Triton (which uses H100-specific instructions), as well as a vendor's implementation of FlashAttention-2 optimized for H100 GPUs from cuDNN. We confirm that FlashAttention-3 is up to 2.0× faster than FlashAttention-2 and 1.5× faster than FlashAttention-2 in Triton. FlashAttention-3 reaches up to 740 TFLOPs/s, 75% of the theoretical maximum TFLOPs/s on H100 GPUs.

- **Ablation study.** We confirm that our algorithmic improvements with warp-specialization and GEMM-softmax pipelining contribute to the speedup of FlashAttention-3.

- **Accuracy of FP8 attention.** We validate that block quantization and incoherent processing reduces the numerical error of FP8 FlashAttention-3 by 2.6×.

## 4.1 Benchmarking Attention

We measure the runtime of different attention methods on an H100 80GB SXM5 GPU for different settings (without / with causal mask, head dimension 64 or 128) for FP16 inputs. We report the results in Fig. 5 and Fig. 6, showing that FlashAttention-3 is around 1.5-2.0× faster than FlashAttention-2 in the forward pass and 1.5-1.75× faster in the backward pass. Compared to a standard attention implementation, FlashAttention-3 can be up to 3-16× faster. For medium and long sequences (1k and above), FlashAttention-3 even surpasses the speed of a vendor's library (cuDNN – closed source) that has been optimized for H100 GPUs.

**Benchmark settings:** We vary the sequence length as 512, 1k, ..., 16k, and set batch size so that the total number of tokens is 16k. We set the hidden dimension to 2048, and head dimension to be either 64, 128, or 256 (i.e., 32 heads, 16 heads, or 8 heads). To calculate the FLOPs of the forward pass, we use:

$$4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}.$$

With causal masking, we divide this number by 2 to account for the fact that approximately only half of the entries are calculated. To get the FLOPs of the backward pass, we multiply the forward pass FLOPs by 2.5 (since there are 2 matmuls in the forward pass and 5 matmuls in the backward pass, due to recomputation).

We also measure the runtime for FP8 for the forward pass under similar settings. We report the results for headdim 256 in Fig. 7 and give the full results in Appendix C.2.

## 4.2 Ablation Study: 2-Stage Pipelining Experiments

We ablate both the 2-stage WGMMA-softmax pipelining and warp-specialization for non-causal FP16 FlashAttention-3 with fixed parameters $\{\text{batch}, \text{seqlen}, \text{nheads}, \text{hdim}\} = \{4, 8448, 16, 128\}$. The result in Table 2 confirms that our algorithmic improvements (asynchrony with warp-specialization and overlapping between GEMM and softmax) lead to significant speedup, from 570 to 661 TFLOPs.



2. **Incoherent processing**: to even out outliers, we multiply $\mathbf{Q}$ and $\mathbf{K}$ with a random orthogonal matrix $\mathbf{M}$ before quantizing to FP8. Since $\mathbf{M}$ is orthogonal, $\mathbf{M}\mathbf{M}^\top = I$ and so $(\mathbf{Q}\mathbf{M})(\mathbf{K}\mathbf{M})^\top = \mathbf{Q}\mathbf{K}^\top$, i.e., multiplying both $\mathbf{Q}$ and $\mathbf{K}$ with $\mathbf{M}$ does not change the attention output. This serves to "spread out" the outliers since each entry of $\mathbf{Q}\mathbf{M}$ or $\mathbf{K}\mathbf{M}$ is a random sum of entries of $\mathbf{Q}$ or $\mathbf{K}$, thus reducing quantization error. In practice, we follow Chee et al. [9] and Tseng et al. [58] and choose $\mathbf{M}$ to be the product of random diagonal matrices of $\pm 1$ and a Hadamard matrix, which can be multiplied in $O(d \log d)$ instead of $O(d^2)$, and can also be fused with the rotary embedding at no extra computation cost.

在4.3节中，我们验证了这两种方法最多可将数值误差降低2.6× 。 .

# 4项实证验证

我们利用CUTLASS [57] 中的基础操作，如WGMMA和TMA抽象接口，来实现FlashAttention-3，并对其效率与精度进行评估。

• **注意力机制基准测试**。我们测量了不同序列长度下FlashAttention-3的运行时间，并将其与PyTorch中的标准实现、FlashAttention-2、在Triton中运行的FlashAttention-2（该版本使用针对H100芯片的专用指令），以及由cuDNN为H100显卡优化的FlashAttention-2商业实现进行了对比。测试结果显示，FlashAttention-3的速度比FlashAttention-2快达2.0× 倍，比在Triton中运行的FlashAttention-2快1.5× 倍。此外，FlashAttention-3的浮点运算次数可达740 TFLOPs/秒，这一数值占H100显卡理论最大TFLOPs/秒值的75%。

• **消融实验**。我们证实，通过指令级专化技术和GEMM-Softmax流水线技术实现的算法优化，确实有助于提升FlashAttention-3的运行速度。

• **FP8格式下FlashAttention的精度**。我们通过实验验证，分块量化与非相干处理技术能够将FP8 FlashAttention-3的数值误差降低2.6×。

## 4.1 注意力机制的基准测试

我们测试了在FP16格式输入、不同配置下（无因果掩码/有因果掩码、注意力头维度为64或128）的各种注意力机制在H100 80GB SXM5显卡上的运行时间。相关结果展示在图5和图6中，可以看出FlashAttention-3在前向传播时的速度比FlashAttention-2快约1.5-2.0× 倍，在后向传播时的速度则快1.5-1.75×倍。与标准注意力机制相比，FlashAttention-3的速度甚至可快达3-16× 倍。对于中等长度及更长的序列（1千元素及以上），FlashAttention-3的速度甚至超过了专为H100显卡优化的商业库（即闭源的cuDNN库）的运行速度。

**基准测试设置：**我们将序列长度设置为512、1千、……、16千，并调整批次大小以确保标记数的总和为16千。隐藏维度被设定为2048，而注意力头维度则分别为64、128或256（即对应32个注意力头、16个注意力头或8个注意力头）。为了计算前向传播的浮点运算次数，我们采用以下公式：

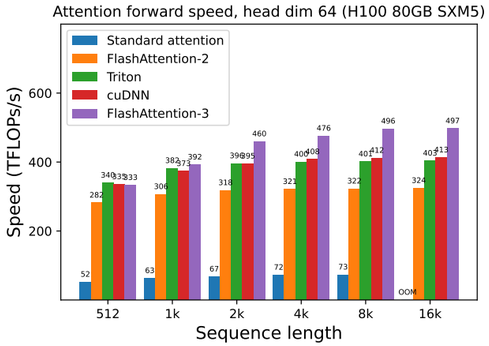
$$4 \cdot \text{seqlen}^2 \cdot \text{头维度} \cdot \text{头数}。$$

在采用因果掩码机制时，我们需要将这个数值除以2，因为实际上只有大约一半的元素会被计算。而要得出后向传播的浮点运算次数，需将前向传播的浮点运算次数乘以2.5（这是因为前向传播中有2次矩阵乘法操作，而后向传播由于需要重新计算，共有5次矩阵乘法操作）。

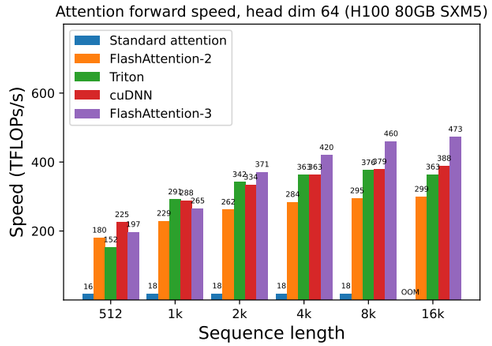
在相同的设置下，我们还测定了FP8格式下进行前向传播时的运行时间。图7展示了headdim为256时的测试结果，而全部测试数据则记载于附录C.2中。

## 4.2 消融研究：两阶段流水线实验

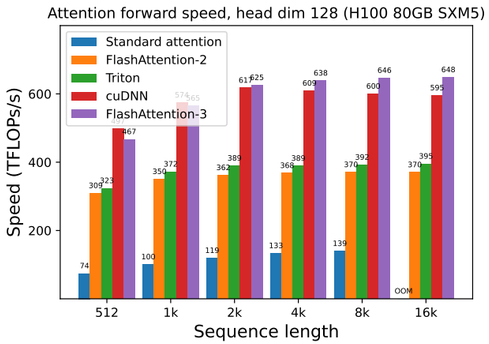
我们对固定参数为$\{\text{batch}, \text{seqlen}, \text{nheads}, \text{hdim}\} = \{4, 8448, 16, 128\}$的非因果FP16 FlashAttention-3架构，同时去除了两阶段的WGMMA-softmax流水线算法以及指令级专化技术。表2中的测试结果表明，我们所采用的算法优化措施——包括结合指令级专化技术的异步处理，以及让GEMM运算与softmax函数重叠执行——带来了显著的性能提升，浮点运算次数从570上升到了661 TFLOPs。



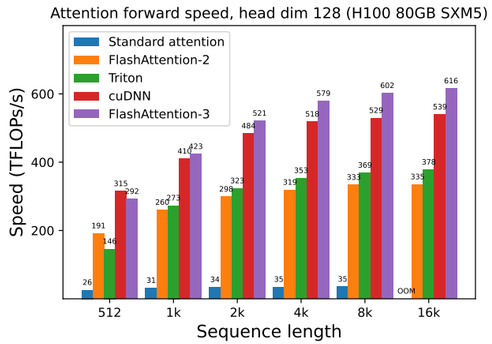

(a) Forward, without causal mask, head dim 64



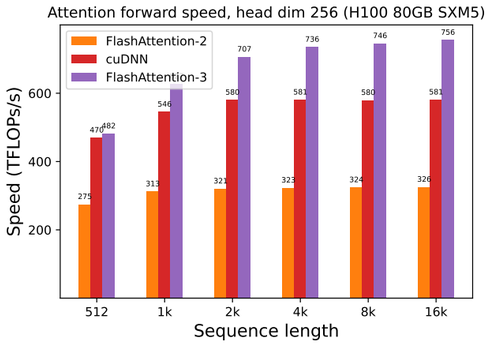

(b) Forward, with causal mask, head dim 64



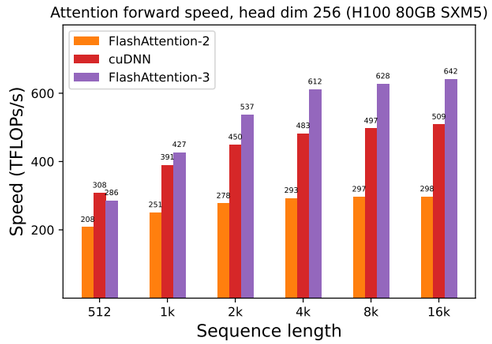

(c) Forward, without causal mask, head dim 128




(d) Forward, with causal mask, head dim 128




(e) Forward, without causal mask, head dim 256



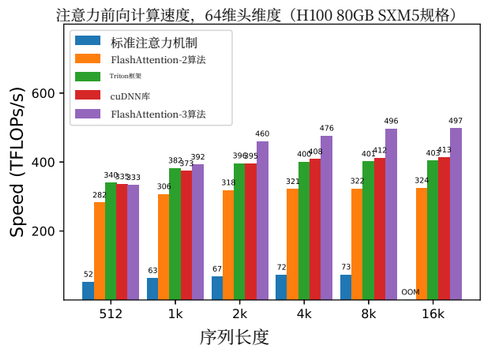

(f) Forward, with causal mask, head dim 256

Figure 5: Attention forward speed (FP16/BF16) on H100 GPU

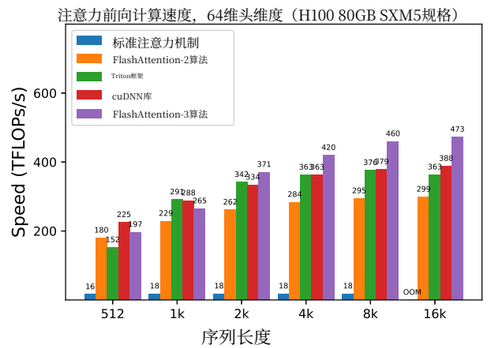
### 4.3 Numerical Error Validation

As there has been interest in the numerical error [21] of FlashAttention, we compare FlashAttention-2, FlashAttention-3, and a standard implementation of attention against a reference implementation in FP64. To simulate outlier features and activations in LLMs [20, 54], we generate the entries of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ with the following

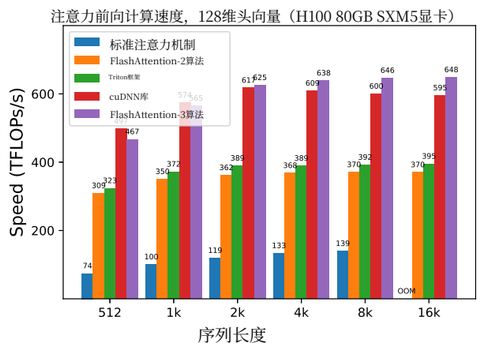




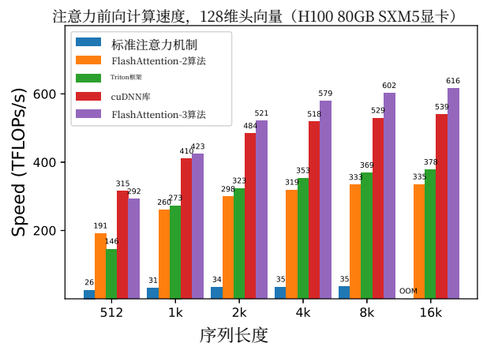

(a) 无因果掩码时的前向传播，注意力头维度为64



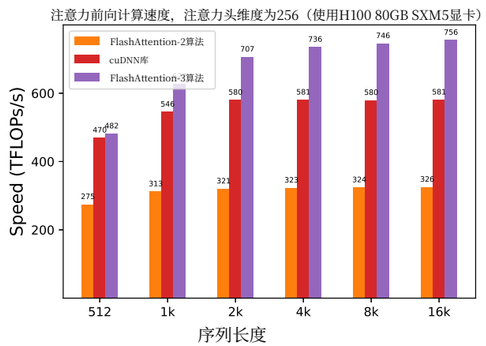

(b) 有因果掩码时的前向传播，注意力头维度为64



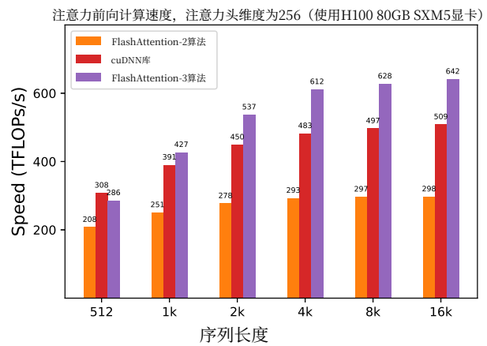

(c) 无因果掩码时的前向传播，注意力头维度为128




(d) 有因果掩码时的前向传播，注意力头维度为128




(e) 在不使用因果掩码的情况下进行前向传播，注意力头维度为256。

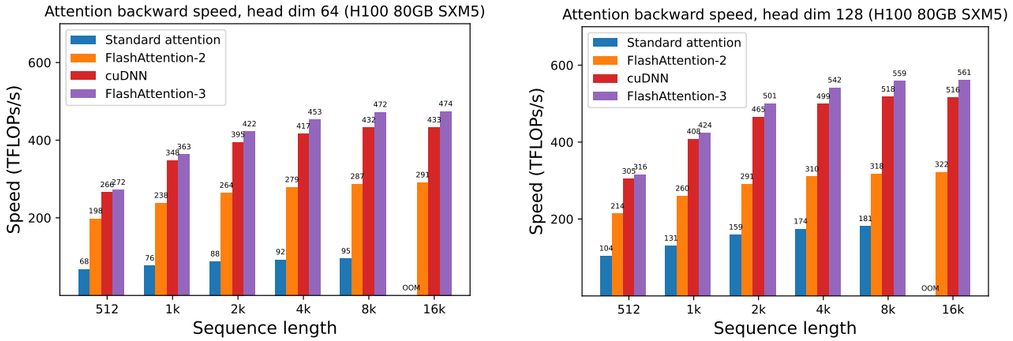



(f) 在使用因果掩码的情况下进行前向传播，注意力头维度为256。

图5：在H100显卡上采用FP16/BF16精度时的注意力前向计算速度。

### 4.3 数值误差验证

由于人们对FlashAttention的数值误差 [21] 很感兴趣，我们便将FlashAttention-2、FlashAttention-3以及标准的注意力实现方式与FP64格式下的参考实现进行了对比。为模拟大型语言模型中的异常特征与激活状态 [20, 54]，，我们采用了以下方式来生成$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$ 这些矩阵的元素。

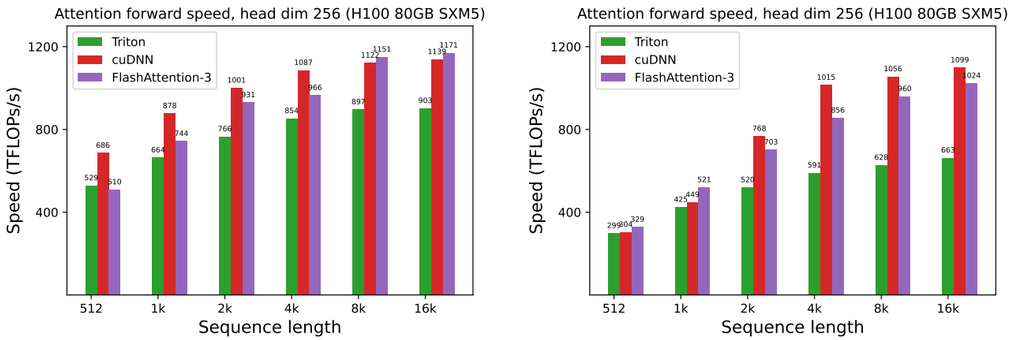



(a) Backward, without causal mask, head dim 64




(b) Backward, without causal mask, head dim 128

Figure 6: Attention backward speed (FP16/BF16) on H100 GPU




(a) Forward, without causal mask, head dim 256




(b) Forward, with causal mask, head dim 256

Figure 7: Attention forward speed (FP8) on H100 GPU

Table 2: Pipelining ablation measurements

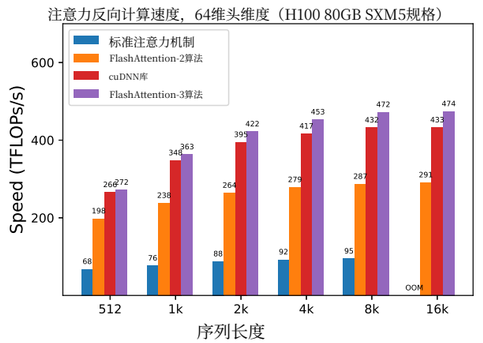
| **Configuration** | **Time** | **TFLOPs/s** |
|---|---|---|
| FlashAttention-3 | 3.538 ms | 661 |
| No GEMM-Softmax Pipelining, Warp-Specialization | 4.021 ms | 582 |
| GEMM-Softmax Pipelining, No Warp-Specialization | 4.105 ms | 570 |

distribution:

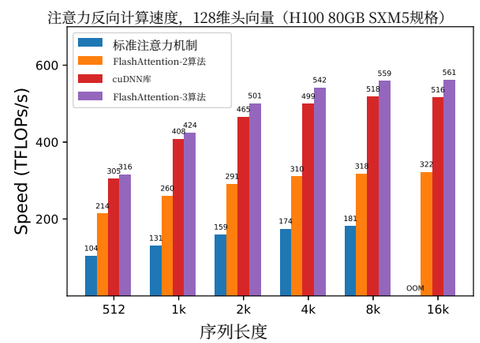
$$\mathcal{N}(0,1) + \mathcal{N}(0,100) \cdot \text{Bernoulli}(0.001).$$

That is, each entry is normally distributed with zero mean and standard deviation 1, but for 0.1% of entries we add an independent term that's normally distributed with standard deviation 10. We then measure the root mean squared error (RMSE) in Table 3. In FP16, both FlashAttention-2 and FlashAttention-3 achieves 1.7× lower RMSE compared to the standard implementation since intermediate results (softmax) are kept in FP32. The baseline attention in FP8 uses per-tensor scaling, with matmul accumulator in FP32 and intermediate softmax results kept in FP16. Thanks to block quantization and incoherent processing, FlashAttention-3 in FP8 is 2.6× more accurate than this baseline.



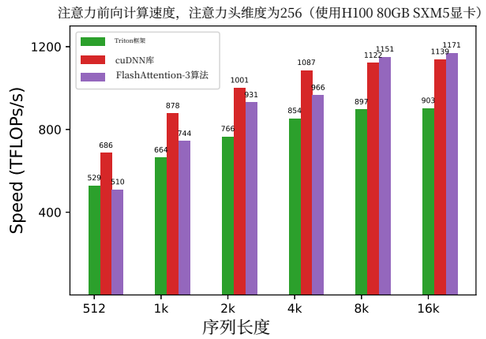



(a) 后向传播，无因果掩码，64维头维度

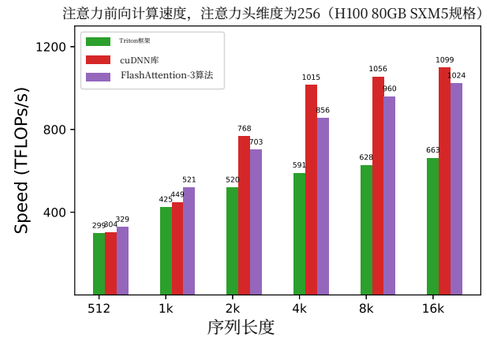



(b) 后向传播，无因果掩码，128维头维度

图6：在H100显卡上使用FP16/BF16精度时的注意力反向计算速度




(a) 无因果掩码的前向传播，注意力头维度为256




(b) 有因果掩码的前向传播，注意力头维度为256

图7：在H100显卡上使用FP8格式时的注意力前向计算速度

表2：流水线消融实验结果

| **配置参数** | **Time** | **TFLOPs/秒** |
|---|---|---|
| FlashAttention-3算法 | 3.538毫秒 | 661 |
| 未启用GEMM-Softmax流水线技术及指令级专化技术 | 4.021毫秒 | 582 |
| 已启用GEMM-Softmax流水线技术，但未采用指令级专化技术 | 4.105 毫秒 | 570 |

分布：

$$\mathcal{N}(0,1) + \mathcal{N}(0,100) \cdot \text{Bernoulli}(0.001).$$

也就是说，每个数据项通常都遵循均值为零、标准差为1的正态分布，但对于0.1%的数据项，我们会额外加入一个标准差为10的独立随机项。随后我们在表3中测算均方根误差（RMSE）。在FP16格式下，由于中间计算结果（即soft max函数的结果）都保存在FP32格式中，FlashAttention-2和FlashAttention-3的RMSE均比标准实现低1.7×。而基于FP8格式的基准注意力机制则采用张量级缩放技术，其中矩阵乘法操作的结果累加器保存在FP32格式中，中间的softmax计算结果则保存在FP16格式中。得益于分块量化与非相干处理技术，FP8格式下的FlashAttention-3精度比这一基准方案高出2.6×。

Table 3: Numerical error comparisons in FP16 and FP8 (e4m3).

| Method | Baseline FP16 | FlashAttention-2 FP16 | FlashAttention-3 FP16 |
|---|---|---|---|
| RMSE | 3.2e-4 | **1.9e-4** | **1.9e-4** |

| Method | Baseline FP8 | FlashAttention-3 FP8 | No block quant | No incoherent processing |
|---|---|---|---|---|
| RMSE | 2.4e-2 | **9.1e-3** | 9.3e-3 | 2.4e-2 |

# 5 Dicussion, Limitations, Conclusion

With FlashAttention-3, we have demonstrated that new programming techniques and hardware features such as asynchrony and low-precision can have a dramatic impact on the efficiency and accuracy of attention. We are able to speed up attention by 1.5-2.0× times compared to FlashAttention-2, and reduce FP8 numerical error by 2.6× compared to standard per-tensor quantization. Some limitations of our work that we hope to address in the future include: optimizing for LLM inference, integrating a persistent kernel design into the FP8 kernel,[10] and understanding the effects of low-precision attention in large-scale training. Though we have focused on Hopper GPUs in this work, we expect that the techniques developed here will apply to other hardware accelerators. We hope that a faster and more accurate primitive such as attention will unlock new applications in long-context tasks.


### Acknowledgments

We are grateful to the NVIDIA CUTLASS team (especially Haicheng Wu, Aniket Shivam, and Cris Cecka) for helping us understand Hopper's programming model and for their library, which provides clean and powerful building blocks for the implementation of FlashAttention-3. We thank the cuDNN team for the idea of in-kernel transpose for FP8. The idea of overlapping GEMMs and softmax was inspired by insightful conversations with Christopher Ré, Benjamin Spector, Aniket Shivam, and Markus Hoehnerbach. The pingpong scheduling is adapted from the warp-specialized pingpong GEMM implementation in CUTLASS. We appreciate Driss Guessous for integrating FlashAttention to PyTorch. FlashAttention-3 has benefited from helpful discussions with Horace He on different attention variants, with Hao Liu and Phil Wang on distributed attention, and with Daniel Haziza and Chris De Sa on quantization. We thank Meta, Together AI, and Princeton Language and Intelligence (PLI) for compute support.


## References


[1] Ahmad Abdelfattah, Azzam Haidar, Stanimire Tomov, and Jack Dongarra. Performance, design, and autotuning of batched gemm for gpus. pages 21–38, 06 2016. ISBN 978-3-319-41320-4. doi: 10.1007/978-3-319-41321-1_2.

[2] AI21. Introducing jamba: Ai21's groundbreaking ssm-transformer model. *AI21 blog*, 2024.

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

[4] Michael Bauer, Henry Cook, and Brucek Khailany. CudaDMA: Optimizing GPU Memory Bandwidth via Warp Specialization. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '11, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450307710. doi: 10.1145/2063384.2063400. URL `https://doi.org/10.1145/2063384.2063400`.

[5] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.


[10] For our benchmarks, FP16 FlashAttention-3 has a persistent kernel and load balancing strategy, while FP8 FlashAttention-3 does not. This partly explains why FP8 FlashAttention-3 does not perform as well for small sequence length and causal masking compared to the FP8 cuDNN kernels.



表3：FP16格式与FP8格式（e4m3精度等级）下的数值误差对比。

| 方法 | 基准FP16 | FlashAttention-2算法的FP16版本 | FlashAttention-3算法的FP16版本 |
|---|---|---|---|
| RMSE | 3.2e-4 | **1.9e-4** | **1.9e-4** |

| 方法 | 基准方案：FP8格式 | 基准方案：FP8 FlashAttention-3算法 | 无块级量化 | 未采用非相干处理技术。 |
|---|---|---|---|---|
| RMSE | 2.4e-2 | **9.1e-3** | 9.3e-3 | 2.4e-2 |

# 5 讨论、局限性及结论

借助FlashAttention-3，我们证明了诸如异步处理和低精度这类新的编程技术及硬件特性，能够对注意力机制的效率与准确性产生显著影响。与FlashAttention-2相比，我们成功将注意力机制的运算速度提升了1.5到2.0× 倍；与标准的张量级量化方法相比，则将FP8格式下的数值误差降低了2.6× 。目前我们的研究还存在一些局限性，未来希望加以解决，包括：针对大语言模型推理进行优化、将持久化内核设计整合到FP8内核中[10] ，以及深入研究低精度注意力机制在大规模训练中的影响。尽管本项研究以Hopper架构的GPU为研究对象，但我们认为此处开发出的技术同样适用于其他硬件加速器。我们期望，像注意力机制这样更快速、更精准的基础算子，能为长上下文任务带来全新的应用可能。


### 致谢

We are grateful to the NVIDIA CUTLASS team (especially Haicheng Wu, Aniket Shivam, and Cris Cecka) for helping us understand Hopper's programming model and for their library, which provides clean and powerful building blocks for the implementation of FlashAttention-3. We thank the cuDNN team for the idea of in-kernel transpose for FP8. The idea of overlapping GEMMs and softmax was inspired by insightful conversations with Christopher Ré, Benjamin Spector, Aniket Shivam, and Markus Hoehnerbach. The pingpong scheduling is adapted from the warp-specialized pingpong GEMM implementation in CUTLASS. We appreciate Driss Guessous for integrating FlashAttention to PyTorch. FlashAttention-3 has benefited from helpful discussions with Horace He on different attention variants, with Hao Liu and Phil Wang on distributed attention, and with Daniel Haziza and Chris De Sa on quantization. We thank Meta, Together AI, and Princeton Language and Intelligence (PLI) for compute support.



## References



[1] Ahmad Abdelfattah, Azzam Haidar, Stanimire Tomov, and Jack Dongarra. Performance, design, and autotuning of batched gemm for gpus. pages 21–38, 06 2016. ISBN 978-3-319-41320-4. doi: 10.1007/978-3-319-41321-1_2.

[2] AI21. Introducing jamba: Ai21's groundbreaking ssm-transformer model. *AI21 blog*, 2024.

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

[4] Michael Bauer, Henry Cook, and Brucek Khailany. CudaDMA: Optimizing GPU Memory Bandwidth via Warp Specialization. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '11, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450307710. doi: 10.1145/2063384.2063400. URL `https://doi.org/10.1145/2063384.2063400`.

[5] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.


[10] 在我们的基准测试中，FP16 FlashAttention-3算法配备了固定的内核以及负载均衡策略，而FP8 FlashAttention-3则不具备这些功能。这也在一定程度上解释了为何在处理序列长度较短且需要应用因果掩码的场景时，FP8 FlashAttention-3的性能会逊于基于FP8的cuDNN内核。

[6] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[7] Ganesh Bikshandi and Jay Shah. Delivering 1 PFLOP/s of Performance with FP8 FlashAttention-2, 2024. URL `https://research.colfax-intl.com/adding-fp8-to-flashattention/`.

[8] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped attention: Faster ring attention for causal transformers. *arXiv preprint arXiv:2311.09431*, 2023.

[9] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. Quip: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*, 36, 2024.

[10] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[11] Richard J Chen, Chengkuan Chen, Yicong Li, Tiffany Y Chen, Andrew D Trister, Rahul G Krishnan, and Faisal Mahmood. Scaling vision transformers to gigapixel images via hierarchical self-supervised learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16144–16155, 2022.

[12] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[13] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *The International Conference on Learning Representations (ICLR)*, 2021.

[14] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.

[15] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, 2023. URL `https://arxiv.org/abs/2307.08691`.

[16] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms with structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.

[17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[18] Tri Dao, Daniel Y Fu, Khaled K Saab, Armin W Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models. In *The International Conference on Learning Representations (ICLR)*, 2023.

[19] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[20] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm. int8 (): 8-bit matrix multiplication for transformers at scale. *CoRR abs/2208.07339*, 2022.

[21] Alicia Golden, Samuel Hsia, Fei Sun, Bilge Acun, Basil Hosmer, Yejin Lee, Zachary DeVito, Jeff Johnson, Gu-Yeon Wei, David Brooks, et al. Is flash attention stable? *arXiv preprint arXiv:2405.02803*, 2024.

[22] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. 2023.

[23] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.



[6] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[7] Ganesh Bikshandi and Jay Shah. Delivering 1 PFLOP/s of Performance with FP8 FlashAttention-2, 2024. URL `https://research.colfax-intl.com/adding-fp8-to-flashattention/`.

[8] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped attention: Faster ring attention for causal transformers. *arXiv preprint arXiv:2311.09431*, 2023.

[9] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. Quip: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*, 36, 2024.

[10] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[11] Richard J Chen, Chengkuan Chen, Yicong Li, Tiffany Y Chen, Andrew D Trister, Rahul G Krishnan, and Faisal Mahmood. Scaling vision transformers to gigapixel images via hierarchical self-supervised learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16144–16155, 2022.

[12] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[13] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *The International Conference on Learning Representations (ICLR)*, 2021.

[14] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.

[15] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, 2023. URL `https://arxiv.org/abs/2307.08691`.

[16] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms with structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.

[17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[18] Tri Dao, Daniel Y Fu, Khaled K Saab, Armin W Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models. In *The International Conference on Learning Representations (ICLR)*, 2023.

[19] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[20] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm. int8 (): 8-bit matrix multiplication for transformers at scale. *CoRR abs/2208.07339*, 2022.

[21] Alicia Golden, Samuel Hsia, Fei Sun, Bilge Acun, Basil Hosmer, Yejin Lee, Zachary DeVito, Jeff Johnson, Gu-Yeon Wei, David Brooks, et al. Is flash attention stable? *arXiv preprint arXiv:2405.02803*, 2024.

[22] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. 2023.

[23] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.

[24] Mandy Guo, Joshua Ainslie, David Uthus, Santiago Ontanon, Jianmo Ni, Yun-Hsuan Sung, and Yinfei Yang. Longt5: Efficient text-to-text transformer for long sequences. *arXiv preprint arXiv:2112.07916*, 2021.

[25] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *Advances in Neural Information Processing Systems*, 35:8633–8646, 2022.

[26] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[27] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

[28] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[30] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[31] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.

[32] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with ringattention. *arXiv preprint arXiv:2402.08268*, 2024.

[33] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

[34] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Qiang Wang, and Xiaowen Chu. Benchmarking and Dissecting the Nvidia Hopper GPU Architecture, 2024. URL `https://arxiv.org/abs/2402.13499`.

[35] Xuezhe Ma, Chunting Zhou, Xiang Kong, Junxian He, Liangke Gui, Graham Neubig, Jonathan May, and Luke Zettlemoyer. Mega: Moving average equipped gated attention. In *The International Conference on Learning Representations (ICLR)*, 2023.

[36] Xuezhe Ma, Xiaomeng Yang, Wenhan Xiong, Beidi Chen, Lili Yu, Hao Zhang, Jonathan May, Luke Zettlemoyer, Omer Levy, and Chunting Zhou. Megalodon: Efficient llm pretraining and inference with unlimited context length. *arXiv preprint arXiv:2404.08801*, 2024.

[37] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[38] NVIDIA. CUDA Programming Guide Version 12.4, 2024. URL `https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html`.

[39] Nvidia. Accelerating transformers with nvidia cudnn 9. *Nvidia blog*, 2024. URL `https://developer.nvidia.com/blog/accelerating-transformers-with-nvidia-cudnn-9/`.

[40] NVIDIA. Parallel Thread Execution ISA Version 8.4, 2024. URL `https://docs.nvidia.com/cuda/pdf/ptx_isa_8.4.pdf`.



[24] Mandy Guo, Joshua Ainslie, David Uthus, Santiago Ontanon, Jianmo Ni, Yun-Hsuan Sung, and Yinfei Yang. Longt5: Efficient text-to-text transformer for long sequences. *arXiv preprint arXiv:2112.07916*, 2021.

[25] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *Advances in Neural Information Processing Systems*, 35:8633–8646, 2022.

[26] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[27] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

[28] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[30] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[31] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.

[32] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with ringattention. *arXiv preprint arXiv:2402.08268*, 2024.

[33] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

[34] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Qiang Wang, and Xiaowen Chu. Benchmarking and Dissecting the Nvidia Hopper GPU Architecture, 2024. URL `https://arxiv.org/abs/2402.13499`.

[35] Xuezhe Ma, Chunting Zhou, Xiang Kong, Junxian He, Liangke Gui, Graham Neubig, Jonathan May, and Luke Zettlemoyer. Mega: Moving average equipped gated attention. In *The International Conference on Learning Representations (ICLR)*, 2023.

[36] Xuezhe Ma, Xiaomeng Yang, Wenhan Xiong, Beidi Chen, Lili Yu, Hao Zhang, Jonathan May, Luke Zettlemoyer, Omer Levy, and Chunting Zhou. Megalodon: Efficient llm pretraining and inference with unlimited context length. *arXiv preprint arXiv:2404.08801*, 2024.

[37] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[38] NVIDIA. CUDA Programming Guide Version 12.4, 2024. URL `https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html`.

[39] Nvidia. Accelerating transformers with nvidia cudnn 9. *Nvidia blog*, 2024. URL `https://developer.nvidia.com/blog/accelerating-transformers-with-nvidia-cudnn-9/`.

[40] NVIDIA. Parallel Thread Execution ISA Version 8.4, 2024. URL `https://docs.nvidia.com/cuda/pdf/ptx_isa_8.4.pdf`.

[41] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, PPoPP '23, pages 429–431, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700156. doi: 10.1145/3572848.3577479. URL https://doi.org/10.1145/3572848.3577479.

[42] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, et al. RWKV: Reinventing RNNs for the Transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

[43] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

[44] Hao Peng, Nikolaos Pappas, Dani Yogatama, Roy Schwartz, Noah A Smith, and Lingpeng Kong. Random feature attention. In *The International Conference on Learning Representations (ICLR)*, 2021.

[45] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[46] Colfax Research. Tutorial: Matrix Transpose in CUTLASS, 2024. URL https://research.colfax-intl.com/tutorial-matrix-transpose-in-cutlass/.

[47] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing Transformers. *arXiv preprint arXiv:2003.05997*, 2020.

[48] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[49] Rya Sanovar, Srikant Bharadwaj, Renee St. Amant, Victor Rühle, and Saravan Rajmohan. Lean attention: Hardware-aware scalable attention mechanism for the decode-phase of transformers. 2024.

[50] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, et al. Scrolls: Standardized comparison over long language sequences. *arXiv preprint arXiv:2201.03533*, 2022.

[51] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[52] Benjamin Spector, Aaryan Singhal, Simran Arora, and Christopher Ré, 2024. URL https://github.com/HazyResearch/ThunderKittens.

[53] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 1441–1450, 2019.

[54] Mingjie Sun, Xinlei Chen, J Zico Kolter, and Zhuang Liu. Massive activations in large language models. *arXiv preprint arXiv:2402.17762*, 2024.

[55] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[56] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.

[57] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS, January 2023. URL https://github.com/NVIDIA/cutlass.



[41] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, PPoPP '23, pages 429–431, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700156. doi: 10.1145/3572848.3577479. URL https://doi.org/10.1145/3572848.3577479.

[42] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, et al. RWKV: Reinventing RNNs for the Transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

[43] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

[44] Hao Peng, Nikolaos Pappas, Dani Yogatama, Roy Schwartz, Noah A Smith, and Lingpeng Kong. Random feature attention. In *The International Conference on Learning Representations (ICLR)*, 2021.

[45] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[46] Colfax Research. Tutorial: Matrix Transpose in CUTLASS, 2024. URL https://research.colfax-intl.com/tutorial-matrix-transpose-in-cutlass/.

[47] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing Transformers. *arXiv preprint arXiv:2003.05997*, 2020.

[48] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[49] Rya Sanovar, Srikant Bharadwaj, Renee St. Amant, Victor Rühle, and Saravan Rajmohan. Lean attention: Hardware-aware scalable attention mechanism for the decode-phase of transformers. 2024.

[50] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, et al. Scrolls: Standardized comparison over long language sequences. *arXiv preprint arXiv:2201.03533*, 2022.

[51] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[52] Benjamin Spector, Aaryan Singhal, Simran Arora, and Christopher Ré, 2024. URL https://github.com/HazyResearch/ThunderKittens.

[53] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 1441–1450, 2019.

[54] Mingjie Sun, Xinlei Chen, J Zico Kolter, and Zhuang Liu. Massive activations in large language models. *arXiv preprint arXiv:2402.17762*, 2024.

[55] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[56] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.

[57] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS, January 2023. URL https://github.com/NVIDIA/cutlass.

[58] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks. *arXiv preprint arXiv:2402.04396*, 2024.

[59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[60] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, et al. An empirical study of mamba-based language models. *arXiv preprint arXiv:2406.07887*, 2024.

[61] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence. AAAI Conference on Artificial Intelligence*, volume 35, page 14138, 2021.

[62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[63] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[64] Zyphra. Zyphra unveils zamba: A compact 7b ssm hybrid model. *Zyphra blog*, 2024.



[58] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks. *arXiv preprint arXiv:2402.04396*, 2024.

[59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[60] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, et al. An empirical study of mamba-based language models. *arXiv preprint arXiv:2406.07887*, 2024.

[61] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence. AAAI Conference on Artificial Intelligence*, volume 35, page 14138, 2021.

[62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[63] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[64] Zyphra。Zyphra推出了zamba：一款体积小巧的70亿参数SSM混合模型。*Zyphra*博客，2024年。

16

# A Related Work

**Attention variants and distributed attention** Ever since attention became popular with the Transformer architecture [59], there has been a large body of work on approximating attention to scale it to longer sequences. These approximation methods can generally be categorized into two classes: sparse and low-rank. Sparse attention only computes some entries of the attention matrix (softmax($\mathbf{QK}^T$)) and assumes that other entries are zero. Different methods have different ways of choosing which entries should be zero, either with a fixed pattern [12], with a sliding window [6], or with a dynamic pattern through hashing [28] or routing [47]. The low-rank approach instead assumes that the attention matrix has a low-rank structure, and apply a pointwise nonlinearity to the query and key [27] with random projection [13, 44, 61]. One can also combine the sparse and low-rank approximation for better quality [10, 63]. However, these approximation methods typically do not offer the same model quality as standard attention [56], and so most large-scale models do not employ these techniques.

There are other variants of attention aimed at reducing the size of the KV cache to improve inference efficiency. Multi-query attention [51] and grouped query attention [3] tie different heads of $\mathbf{K}$ and $\mathbf{V}$, and multiple query heads interact with the same key and value head. Multi-head latent attention [19] parameterizes the $\mathbf{K}$ and $\mathbf{V}$ as low-rank projections of a shared matrix to further reduce the KV cache size. However, all of these approaches do not change the core computation softmax($\mathbf{QK}^T$)$\mathbf{V}$ during training and simply change how $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are obtained. As a result, any efficiency or accuracy improvement to the standard attention computation benefits these methods.

To extend to even longer context, attention computation can be distributed across multiple GPUs. Methods such as Ring attention [31, 32] and variants [8] can reach a context length of up to 1 million. They use FlashAttention (or FlashAttention-2) as a primitive, and so the improvement from FlashAttention-3 would benefit these distributed attention methods as well.

**Alternative architectures** Motivated by the limitations of attention, a variety of alternative architectures have been proposed. They build on the connection between linear attention [27] and recurrent neural networks (RNNs). RWKV [42], H3 [18], MEGA [35], Retnet [55] enhance the expressivity of the simple cumulative sum in linear attention with more sophisticated recurrences. Mamba [22] and xLSTM [5] use learnable weighting for the recurrence and can match the quality of Transformers in language modeling at small or medium scale. These approaches can be connected to generalizations of linear attention through the lens of the structure of the token-mixing matrix [16]. These models have started to see some traction, seeing usage in some medium to large-scale models such as Jamba [2], Zamba [64], Megalodon [36], and Mamba2-hybrid [60]. For the highest quality, these SSM- and RNN-based models still employ many layers of attention. We expect that techniques to speed up attention presented in this work will be useful to speedup these alternative architectures.

**Low-precision attention** Quantization is a promising approach to speed up attention, but they have mostly focused on reducing the space for KV cache for inference efficiency. QuIP [9] and QuIP#[58] use incoherent processing to reduce the quantization, and we adapted this technique for FP8 FlashAttention-3. Recent work suggests that for inference the KV cache is highly compressible down to 4-, 3-, or even 2-bits [26, 33]. However, quantization during training is still challenging as higher precision is typically required for stable training.

**Hardware-aware Algorithms** Our work presented in this paper focuses on the micro-architecture specific tuning to leverage new instruction sets and adopt a natively asynchronous programming model. There are other orthogonal axes for hardware-aware algorithm co-design being explored. A recent example of this is LeanAttention [49], which recognizes the poor GPU occupancy and high memory bandwidth requirements of the sequential token generation phase as primary bottlenecks for inference and optimizes it via a smarter load balancing strategy similar to Stream-K load balancing [41] to achieve nearly peak occupancy. There is a large literature on optimizing GEMM for specific hardware that employs many of the same techniques. As an example, Abdelfattah et al. [1] presents a high performance batched GEMM kernel on K40c Graphics Processing Units (GPU) for both fixed and variable sizes, proposing specialized GEMM designs and a comprehensive autotuning process to deliver state-of-the-art performance.



# 一项相关研究

**Attention variants and distributed attention** Ever since attention became popular with the Transformer architecture [59], there has been a large body of work on approximating attention to scale it to longer sequences. These approximation methods can generally be categorized into two classes: sparse and low-rank. Sparse attention only computes some entries of the attention matrix (softmax($\mathbf{QK}^T$)) and assumes that other entries are zero. Different methods have different ways of choosing which entries should be zero, either with a fixed pattern [12], with a sliding window [6], or with a dynamic pattern through hashing [28] or routing [47]. The low-rank approach instead assumes that the attention matrix has a low-rank structure, and apply a pointwise nonlinearity to the query and key [27] with random projection [13, 44, 61]. One can also combine the sparse and low-rank approximation for better quality [10, 63]. However, these approximation methods typically do not offer the same model quality as standard attention [56], and so most large-scale models do not employ these techniques.

There are other variants of attention aimed at reducing the size of the KV cache to improve inference efficiency. Multi-query attention [51] and grouped query attention [3] tie different heads of $\mathbf{K}$ and $\mathbf{V}$, and multiple query heads interact with the same key and value head. Multi-head latent attention [19] parameterizes the $\mathbf{K}$ and $\mathbf{V}$ as low-rank projections of a shared matrix to further reduce the KV cache size. However, all of these approaches do not change the core computation softmax($\mathbf{QK}^T$)$\mathbf{V}$ during training and simply change how $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are obtained. As a result, any efficiency or accuracy improvement to the standard attention computation benefits these methods.

为支持更长的上下文长度，注意力计算可以被分散到多台GPU上执行。诸如Ring attention [31, 32] 及其变体 [8] 之类的方法能够实现高达100万的上下文长度。这类方法以FlashAttention（或FlashAttention-2）作为基础算子，因此FlashAttention-3算法带来的改进同样会惠及这些分布式注意力计算方法。

**Alternative architectures** Motivated by the limitations of attention, a variety of alternative architectures have been proposed. They build on the connection between linear attention [27] and recurrent neural networks (RNNs). RWKV [42], H3 [18], MEGA [35], Retnet [55] enhance the expressivity of the simple cumulative sum in linear attention with more sophisticated recurrences. Mamba [22] and xLSTM [5] use learnable weighting for the recurrence and can match the quality of Transformers in language modeling at small or medium scale. These approaches can be connected to generalizations of linear attention through the lens of the structure of the token-mixing matrix [16]. These models have started to see some traction, seeing usage in some medium to large-scale models such as Jamba [2], Zamba [64], Megalodon [36], and Mamba2-hybrid [60]. For the highest quality, these SSM- and RNN-based models still employ many layers of attention. We expect that techniques to speed up attention presented in this work will be useful to speedup these alternative architectures.

**低精度注意力**量化是一种有望加快注意力计算速度的技术，但目前相关研究大多侧重于通过压缩KV缓存来提升推理效率。QuIP [9] 与QuIP#[58] 采用了非相干处理技术来实现量化降维，我们则将这一技术应用到了FP8 FlashAttention-3算法中。近期有研究指出，在推理场景下KV缓存完全可以被高度压缩至4位、3位甚至2位 [26, 33]。不过在训练阶段进行量化仍然面临诸多挑战，因为稳定的训练过程通常需要更高的精度。

**Hardware-aware Algorithms** Our work presented in this paper focuses on the micro-architecture specific tuning to leverage new instruction sets and adopt a natively asynchronous programming model. There are other orthogonal axes for hardware-aware algorithm co-design being explored. A recent example of this is LeanAttention [49], which recognizes the poor GPU occupancy and high memory bandwidth requirements of the sequential token generation phase as primary bottlenecks for inference and optimizes it via a smarter load balancing strategy similar to Stream-K load balancing [41] to achieve nearly peak occupancy. There is a large literature on optimizing GEMM for specific hardware that employs many of the same techniques. As an example, Abdelfattah et al. [1] presents a high performance batched GEMM kernel on K40c Graphics Processing Units (GPU) for both fixed and variable sizes, proposing specialized GEMM designs and a comprehensive autotuning process to deliver state-of-the-art performance.

# B Addition Details on Algorithms

## B.1 Asynchrony Through Warp Specialization for the Backward Pass

Similar to the forward pass § 3.1, we use warp specialization to handle asynchrony. Instead of just a simple producer-consumer pattern in the forward pass, we add one extra role of a **dQ** writer, since we need to accumulate the value of **dQ** produced by each thread block to the global value of **dQ**. This **dQ** accumulation introduces memory contention (many thread blocks writing to the same location) so having a separate warp to handle this (along with asynchrony) will avoid blocking the rest of the warps in the thread block to perform the next computation (matmul).

We include the backward pass with warp specialization in Algorithm 3.

**Algorithm 3** FlashAttention-3 backward pass with warp specialization

**Require:** Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, logsumexp vector $L \in \mathbb{R}^N$ in HBM, block sizes $B_c$, $B_r$.

In a preprocessing kernel, compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write $D$ to HBM and divide it into $T_r$ blocks $D_1, \ldots, D_{T_r}$ of size $B_r$ each.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{dO}$ into $T_r$ blocks $\mathbf{dO}_i, \ldots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide $L$ into $T_r$ blocks $L_i, \ldots, L_{T_r}$ of size $B_r$ each.
Initialize pipeline object to manage barrier synchronization with $s$-stage circular SMEM buffer.
**if** in producer warpgroup **then**
  Deallocate predetermined number of registers.
  Issue load $\mathbf{K}_j$ and $\mathbf{V}_j$ from HBM to shared memory.
  Upon completion, commit to notify consumer of the load of $\mathbf{K}_j$ and $\mathbf{V}_j$.
  **for** $1 \le i \le T_r$ **do**
    Wait for the $(i \,\%\, s)$th stage of the buffer to be consumed.
    Issue loads of $\mathbf{Q}_i, \mathbf{dO}_i$ from HBM to shared memory at the $(i \,\%\, s)$th stage of the buffer.
    Upon completion, commit to notify consumers of the loads of $\mathbf{Q}_i, \mathbf{dO}_i$.
  **end for**
**else if** in consumer warpgroups **then**
  Reallocate predetermined number of registers as function of number of consumer warps.
  On-chip, Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ .
  Wait for $\mathbf{K}_j$ and $\mathbf{V}_j$ to be loaded in shared memory.
  **for** $1 \le i \le T_r$ **do**
    Wait for $\mathbf{Q}_i$ to be loaded in shared memory.
    Load $L_i, D_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ (SS-GEMM). Commit.
    Wait for $\mathbf{dO}_i$ to be loaded in shared memory.
    On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$ (SS-GEMM). Commit.
    On chip, wait for $\mathbf{S}_i^{(j)}$, then compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, wait for $\mathbf{dP}_i^{(j)}$, then compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$ (RS-GEMM). Commit.
    On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$ (RS-GEMM). Commit and wait for both $\mathbf{dV}_j$ and $\mathbf{dK}_j$.
    On chip, compute $\mathbf{dQ}_i^{(\text{local})} = \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ (SS-GEMM), and write $\mathbf{dQ}_i^{(\text{local})}$ to smem. Notify the **dQ**-writer.
  **end for**
**else if** in **dQ**-writer warp **then**
  **for** $1 \le i \le T_r$ **do**
    Wait for $\mathbf{dQ}_i^{(\text{local})}$ to be ready in smem.
    Using a semaphore, atomically add $\mathbf{dQ}_i^{(\text{local})}$ to $\mathbf{dQ}_i$ in global memory.
  **end for**
**end if**



# 算法中的B加法细节

## B.1 后向传播中的Warp专用化实现异步处理

与§3.1节中的前向传播类似，我们采用warp专项优化技术来实现异步处理。与前向传播中单纯的生产者-消费者模型不同，这里我们还增加了**dQ** 写入者的额外角色，因为我们需要将每个线程块生成的**dQ** 值累加到全局的**dQ**值中。这种**dQ** 值的累加操作会引发内存竞争（多个线程块同时向同一位置写入数据），因此通过专门的warp来处理这一问题（以及异步处理需求），就能避免阻塞线程块中的其他warp进而影响后续的运算（即矩阵乘法）的进行。

我们在算法3中加入了采用Warp专用化的后向传播机制。

**Algorithm 3** FlashAttention-3 backward pass with warp specialization

**Require:** Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, logsumexp vector $L \in \mathbb{R}^N$ in HBM, block sizes $B_c$, $B_r$.

In a preprocessing kernel, compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write $D$ to HBM and divide it into $T_r$ blocks $D_1, \ldots, D_{T_r}$ of size $B_r$ each.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{dO}$ into $T_r$ blocks $\mathbf{dO}_i, \ldots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide $L$ into $T_r$ blocks $L_i, \ldots, L_{T_r}$ of size $B_r$ each.
Initialize pipeline object to manage barrier synchronization with $s$-stage circular SMEM buffer.
**if** in producer warpgroup **then**
  Deallocate predetermined number of registers.
  Issue load $\mathbf{K}_j$ and $\mathbf{V}_j$ from HBM to shared memory.
  Upon completion, commit to notify consumer of the load of $\mathbf{K}_j$ and $\mathbf{V}_j$.
  **for** $1 \le i \le T_r$ **do**
    Wait for the $(i \,\%\, s)$th stage of the buffer to be consumed.
    Issue loads of $\mathbf{Q}_i, \mathbf{dO}_i$ from HBM to shared memory at the $(i \,\%\, s)$th stage of the buffer.
    Upon completion, commit to notify consumers of the loads of $\mathbf{Q}_i, \mathbf{dO}_i$.
  **end for**
**else if** in consumer warpgroups **then**
  Reallocate predetermined number of registers as function of number of consumer warps.
  On-chip, Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ .
  Wait for $\mathbf{K}_j$ and $\mathbf{V}_j$ to be loaded in shared memory.
  **for** $1 \le i \le T_r$ **do**
    Wait for $\mathbf{Q}_i$ to be loaded in shared memory.
    Load $L_i, D_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ (SS-GEMM). Commit.
    Wait for $\mathbf{dO}_i$ to be loaded in shared memory.
    On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$ (SS-GEMM). Commit.
    On chip, wait for $\mathbf{S}_i^{(j)}$, then compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, wait for $\mathbf{dP}_i^{(j)}$, then compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$ (RS-GEMM). Commit.
    On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$ (RS-GEMM). Commit and wait for both $\mathbf{dV}_j$ and $\mathbf{dK}_j$.
    On chip, compute $\mathbf{dQ}_i^{(\text{local})} = \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ (SS-GEMM), and write $\mathbf{dQ}_i^{(\text{local})}$ to smem. Notify the **dQ**-writer.
  **end for**
**else if** in **dQ**-writer warp **then**
  **for** $1 \le i \le T_r$ **do**
    Wait for $\mathbf{dQ}_i^{(\text{local})}$ to be ready in smem.
    Using a semaphore, atomically add $\mathbf{dQ}_i^{(\text{local})}$ to $\mathbf{dQ}_i$ in global memory.
  **end for**
**end if**

## B.2 2-Stage Pipelining SASS Analysis

We give simplified SASS code for the inside of the consumer warpgroup mainloop.

```
// Compute row_max
FMNMX.FTZ R0, R24, R6, !PT ;
SHFL.BFLY PT, R185, R2, 0x2, 0x1f ;
... FMNMX and SHFL.BFLY ...

// Apply exp2 and row_sum. Rescale O.
FMUL.FTZ R2, R4, UR9 ;
MUFU.EX2 R185, R184 ;
FFMA.FTZ R24, R24, UR9, -R6.reuse ;
FADD.FTZ R24, R211, R24 ;
... FMUL, FFMA, FMUL, MUFU.EX2, FADD ...

// FP32 -> FP16 conversion are interleaved with exp2, row_sum and O rescaling.
F2FP.F16.F32.PACK_AB R231, R25, R231 ;
... F2FP, FMUL, MUFU, FFMA, FADD ...

// Start the first WGMMA. Broken down into 8 HGMMAs.
// The first 7 HGMMAs are packed together.
WARPGROUP.ARRIVE ;
HGMMA.64x192x16.F32 R24, gdesc[UR44], RZ, !UPT ;
... HGMMA x 6 ...

// FP32->FP16, exp2, row_sum, O rescaling are interleaved with HGMMA.
F2FP.F16.F32.PACK_AB R214, R214, R187 ;
MUFU.EX2 R234, R5 ;
FADD.FTZ R237, R187, R2 ;
... F2FP, MUFU, FADD ...

// The last HGMMA is issued here. No need to wait.
HGMMA.64x192x16.F32 R24, gdesc[UR44], R24, gsb0 ;

// Start the second WGMMA. Broken down into 12 HGMMAs.
// All 12 HGMMAs are packed together. Not interleaved with other instructions.
WARPGROUP.ARRIVE ;
HGMMA.64x128x16.F32 R120, R228, gdesc[UR8].tnspB, R120 ;
... HGMMA x 10 ...
HGMMA.64x128x16.F32 R120, R184, gdesc[UR8].tnspB, R120, gsb0 ;

// wgmma.wait_group at the end.
WARPGROUP.DEPBAR.LE gsb0, 0x0 ;
```

We make the following observations:

1. Softmax is reordered to the very beginning, even before the first WGMMA.
2. The first WGMMA is interleaved with softmax and FP32 → FP16 datatype conversion of **S**. This indicates that WGMMA and non-WGMMAs are executed in parallel.
3. `exp2`, `row\_sum`, O rescaling and FP32 → FP16 conversions are interleaved together.
4. The second WGMMA is not overlapped with other instructions, as expected.

Overall, SASS shows that the 2-stage pipelining idea works as expected.



## B.2 两阶段流水线SASS分析

我们提供了消费者warpgroup主循环内部的简化版SASS代码。

// 计算行_max：FMNMX.FTZ R0,R24, R6, !
PT；执行SHFL.BFLY指令：PT,R185, R2,
0x2, 0x1f；... FMNMX与SHFL.BFLY指令 ...

// 执行exp2运算与row_sum操作，随后进行
RescaleO处理。FMUL.FTZ R2,R4, UR9 ;
MUFU.EX2 R185, R184 ; FFMA.FTZ R24,R24,
UR9, -R6.reuse ; FADD.FTZ R24,R211, R24 ; ...
FMUL、FFMA、FMUL、MUFU.EX2、FADD ...

// FP32格式与>FP16格式之间的转换操作会与exp2运算、行_求和以及O比例缩放操作交错进行。
F2FP.F16.F32.PACK_AB R231, R25, R231 ; ... F2FP, FMUL, MUFU, FFMA, FADD...

// 启动第一个WGMMA运算，该运算被拆分为8个
HGMMAs。// 前7个HGMMAs会被打包在一起。
WARPGROUP.ARRIVE ; HGMMA.64x192x16.F32 R24,
gdesc[UR44]，RZ, !UPT ; ... HGMMA x6 ...

// FP32->FP16格式、exp2运算、row_求和以及O比例缩放与HGMMA操作相互
交错进行。F2FP.F16.F32.PACK_AB R214, R214, R187；MUFU.EX2 R234,
R5；FADD.FTZ R237, R187, R2；……F2FP、MUFU、FADD……

// 最后的HGMMA指令在此处生成，无需等待。
HGMMA.64x192x16.F32 R24, gdesc[UR44]，R24,gsb0 ;

// 启动第二个WGMMA运算，该运算被拆分为12个HGMMAs。// 这12个HGMMAs会集中
在一起处理，不会与其他指令交错执行。WARPGROUP.ARRIVE ;
HGMMA.64x128x16.F32 R120, R228, gdesc[UR8].tnspB, R120 ; ... 还有10个HGMMA运
算 ... HGMMA.64x128x16.F32 R120, R184, gdesc[UR8].tnspB, R120, gsb0 ;

// 在最后添加wgmma.wait_group。
WARPGROUP.DEPBAR.LE gsb0, 0x0 ;

我们得出以下观察结果：

1. softmax函数被调整到最开头的位置，甚至排在第一个WGMMA运算之前。
2. 第一个WGMMA运算与softmax函数以及**S**值的FP32 → FP16格式转换操作相互交错执行。这表明WGMMA运算与非WGMMA运算是并行处理的。
3. exp2运算、row_sum计算、O比例缩放以及FP32 → FP16格式转换操作也彼此交错进行。er.
4. 如预期，第二个WGMMA运算并未与其他指令发生重叠。

总体而言，SASS的结果表明，两阶段流水线算法的效果符合预期。

## B.3 3-Stage Pipelining Algorithm

We experiment with a 3-stage pipelining algorithm to parallelize the first WGMMA from iteration $j+2$, softmax from iteration $j+1$, and the second WGMMA from iteration $j$. We describe this algorithm in Algorithm 4. This algorithm behaves worse than the 2-stage pipelining algorithm due to the reasons below:

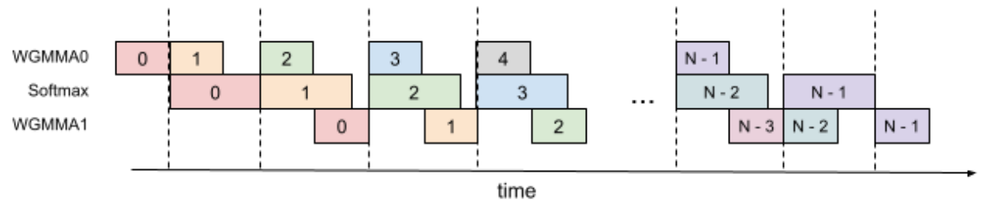



Figure 8: 3-Stage Pipelining

**Algorithm 4** FlashAttention 3-stage pipelining consumer warpgroup forward pass

**Require:** Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes $B_c$, $B_r$. Each warpgroup reads 1 block Qi of size $B_r \times d$, $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$ of size $B_c \times d$. Each warpgroup writes 1 output block $\mathbf{O}_i$ of size $B_r \times d$, and 1 logsumexp block $L_i$ of size $B_r$.

Initialization. Load $\mathbf{Q}_i$ from HBM to on-chip SRAM. Initialize $\mathbf{O}_i, \ell_i, m_i, scale_o$.
Wait for the producer warpgroup loading $\mathbf{K}_0$ from HBM to on-chip SRAM.
Compute $\mathbf{S} = \mathbf{Q}_i \mathbf{K}_0^T$ using WGMMA. Commit and wait.
Compute $m_i$, $\tilde{\mathbf{P}}_i$, $\ell_i$, $scale_o$ based on $\mathbf{S}$.
Wait for the producer warpgroup loading $\mathbf{K}_1$ from HBM to on-chip SRAM.
Compute $\mathbf{S} = \mathbf{Q}_i \mathbf{K}_1^T$ using WGMMA. Commit and wait.
**for** $2 \le j < T_c - 2$ **do**
  Wait for the producer warpgroup loading $\mathbf{K}_j$ from HBM to on-chip SRAM.
  Compute $\mathbf{S}_next = \mathbf{Q}_i \mathbf{K}_j^T$ using WGMMA. Commit but do not wait.
  Wait for the producer warpgroup loading $\mathbf{V}_{j-2}$ from HBM to on-chip SRAM.
  Rescale $\mathbf{O}_i$ based on $scale_o$.
  Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_i \mathbf{V}_{j-2}$ using WGMMA. Commit but do not wait.
  Compute $m_i$, $\tilde{\mathbf{P}}_i_next$, $\ell_i$, $scale_o$ based on $\mathbf{S}$.
  Wait for all previous WGMMAs.
  Copy $\mathbf{S}_next$ to $\mathbf{S}$.
  Copy $\tilde{\mathbf{P}}_i_next$ to $\tilde{\mathbf{P}}_i$.
**end for**
Wait for the producer warpgroup loading $\mathbf{V}_{T_c-2}$ from HBM to on-chip SRAM.
Rescale $\mathbf{O}_i$ based on $scale_o$.
Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_i \mathbf{V}_{T_c-2}$ using WGMMA. Commit and wait.
Compute $m_i$, $\tilde{\mathbf{P}}_i$, $\ell_i$, $scale_o$ based on $\mathbf{S}$.
Wait for the producer warpgroup loading $\mathbf{V}_{T_c-1}$ from HBM to on-chip SRAM.
Rescale $\mathbf{O}_i$ based on $scale_o$.
Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_i \mathbf{V}_{T_c-1}$ using WGMMA. Commit and wait.
Epilogue. Rescale $\mathbf{O}_i$ based on $\ell_i$. Compute $L_i$ based on $\ell_i$ and $m_i$. Write $\mathbf{O}_i$ and $L_i$ to HBM as the $i$-th block of $\mathbf{O}$ and $L$.

**Overlapping.** We expected that softmax can be overlapped with (the first WGMMA + the second WGMMA). However, the compiler doesn't cooperate in this way. SASS code shows that only the first WGMMA is overlapped with softmax, while the second WGMMA is not. It's not clear why the compiler chooses to reorder instructions in this way.



## B.3 三阶段流水线算法

我们尝试采用三阶段流水线算法，将第 $j+2$次迭代中的第一个WGMMA运算、第 $j+1$次迭代中的softmax函数以及第 $j$次迭代中的第二个WGMMA运算进行并行处理。该算法在算法4中有详细说明。由于以下原因，它的性能不如两阶段流水线算法。

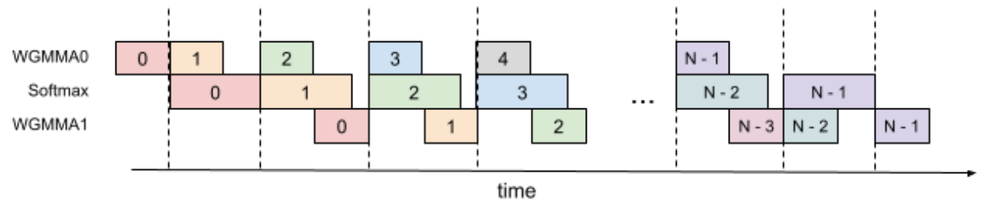



图8：三阶段流水线算法

**算法4** FlashAttention三阶段流水线式消费者线程组前向传播

**所需条件：** 位于高带宽内存中的 $\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$ $\in \mathbb{R}^{N \times d}$矩阵，以及大小为 $B_c$、$B_r$的块尺寸。每个线程组会读取一个大小为 $B_r \times d$的Qi块。$T_c = \left\lceil \frac{N}{B_c} \right\rceil$个块，其结构为**键向量**$_1$,.....，**键向量**$_{T_c}$ 以及**值向量**$_1$,.....，**值向量**$_{T_c}$，所有这些的尺寸均为 $B_c \times d$。每个线程组会写入1个尺寸为该值的**输出结果**$_i$ 块

$B_r \times d$，以及一个大小为 $B_r$的logsumexp运算块 $L_i$。

初始化阶段。将$\mathbf{Q}_i$从高带宽内存加载到片上SRAM中。同时初始化$\mathbf{O}_i$、$l_i$、$m_i$、$scale_o$.
等待生产者线程组将来自高带宽内存的$\mathbf{K}_0$ 数据加载到片上SRAM中。
使用WGMMA运算来计算$\mathbf{S} = \mathbf{Q}_i \mathbf{K}_0^T$，随后提交并等待。
计算 $m_i$，$\tilde{\mathbf{P}}_i$，$l_i$, $scale_o$ 基于$\mathbf{S}$值。
等待生产者线程组将来自高带宽内存的$\mathbf{K}_1$ 数据加载到片上SRAM中。
使用WGMMA运算来计算$\mathbf{S} = \mathbf{Q}_i \mathbf{K}_1^T$，随后提交并等待。
**for** $2 \le j < T_c - 2$ **do**
  等待生产者线程组将$\mathbf{K}j$ 从高带宽内存加载到片上SRAM中。
  使用WGMMA运算计算$\mathbf{S}_next = \mathbf{Q}_i \mathbf{K}_j^T$。执行提交操作，但无需等待。
  等待生产者线程组将$\mathbf{V}_{j-2}$ 从高带宽内存加载到片上SRAM中。
  根据 $scale$对$\mathbf{O}_i$ 进行重新缩放。$_o$.
  计算$\mathbf{O}_i$ =以及$\mathbf{O}_i$ +的值。通过WGMMA运算得到$\mathbf{P}_i$和$\mathbf{V}_{j-2}$，随后立即提交结果而无需等待。
  计算 $m_i$的值。$\tilde{\mathbf{P}}_i_next$, $l_i$, $scale$。$_$基于$\mathbf{S}$来计算$o$。
  等待所有之前的WGMMA运算完成。
  复制<span style id='2'>S值。$_next$ 的值将被复制到<span style id='2'>S中。
  Copy $\tilde{\mathbf{P}}_i_next$ 值将被传输至 $\tilde{\mathbf{P}}_i$.
<span style id='1'>17: 执行结束。

<span style id='1'>18: 等待生产者线程组完成从高带宽内存向片上SRAM加载<span style id='3'>V$_{T_c-2}$ 的数据加载操作。

根据 $scale$对$\mathbf{O}_i$ 进行重新缩放。$_o$.
计算$\mathbf{O}_i$ =$\mathbf{O}_i$ +的值。$\tilde{\mathbf{P}}_i \mathbf{V}_{T_c-2}$，采用WGMMA运算方式。完成操作后请等待。
计算 $m_i$的值。$\tilde{\mathbf{P}}_i$, $l_i$, $scale_o$ 基于$\mathbf{S}$得出。
等待生产者线程组将$\mathbf{V}_{T_c-1}$ 从高带宽内存加载到片上SRAM中。
根据 $scale$对$\mathbf{O}$值$_i$ 进行重新缩放 $_o$.
计算 $\mathbf{O}$值$_i$ =$\mathbf{O}$值$_i$ + $\tilde{\mathbf{P}}_i \mathbf{V}_{T_c-1}$ 通过WGMMA运算完成。提交后等待即可。
尾声阶段。根据$l_i$对$\mathbf{O}_i$ 进行重新缩放。基于$l_i$ 与 $m_i$计算 $L_i$。将$\mathbf{O}_i$ 和 $L_i$ 作为第 $i$个块写入高带宽内存中。$\mathbf{O}$值与 $L$的。

**存在重叠问题**。我们原本期望可以将softmax与（第一个WGMMA +第二个WGMMA）进行重叠处理。然而编译器并不支持这样的操作方式。通过SASS代码可以发现，只有第一个WGMMA能与softmax实现重叠，而第二个WGMMA则无法做到。目前尚不清楚为何编译器会选择以这种方式重新排序指令。

**Register pressure.** This algorithm requires more registers compared to the 2-stage pipelining algorithm. In theory, it needs to store an extra $\tilde{\mathbf{P}}_i$ and $scale_o$, which is of size $B_r \times B_c \times \text{sizeof(input_data_type)} + B_r \times \text{sizeof(float)}$. As a result, a smaller block size needs to be chosen.

# C Addition Details on Experiments and Benchmarking

## C.1 System and libraries

We benchmark the speed on an H100 80GB SXM5 (700W). We generally use the latest versions of the libraries, at the time of writing (May 2024). Specifically, we use:

- CUDA 12.3
- cuDNN 9.1.1.17
- CUTLASS 3.5
- FlashAttention 2.5.8
- Triton nightly 3.0.0.post20240424212437
- PyTorch 2.3.0

To reduce variability, we fix the GPU clock speed to 1830MHz (clock speed used to calculate the 989 TFLOPS FP16 theoretical max throughput). We repeat the benchmarks 100 times and take the average timing.

## C.2 FP8 Attention Full Results

We use following sequence lengths: 512, 1024, 2048, 4224, 8448, 16896. When sequence length $\geq$ 4k, we make it also divisible by 132 (number of SMs in H100 SXM5) to avoid wave quantization.



**寄存器压力问题**。与两阶段流水线算法相比，该算法需要更多的寄存器。从理论上讲，˜它还需要额外存储一个$\mathbf{P}_i$ 以及 $scale_o$，其大小为 $B_r \times B_c \times$sizeof(input_data_type) $+ B_r \times$ sizeof(float)。因此不得不选择更小的块尺寸。

# C项：实验与基准测试中的加法运算细节

## C.1 系统与库

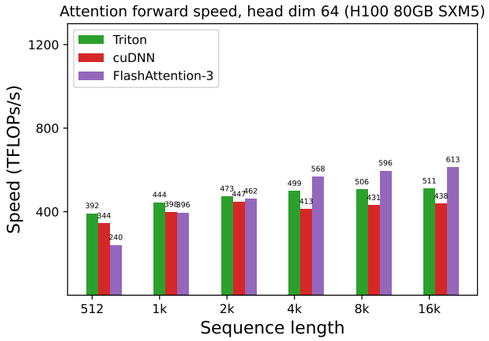
我们在配备H100 80GB SXM5（700W）芯片的硬件上对运算速度进行基准测试。在撰写本文时（2024年5月），我们通常使用各库的最新版本，具体所使用的版本如下：

- CUDA 12.3
- cuDNN 9.1.1.17版本
- CUTLASS 3.5版本
- FlashAttention 2.5.8版本
- Triton夜间版3.0.0.post20240424212437
- PyTorch 2.3.0

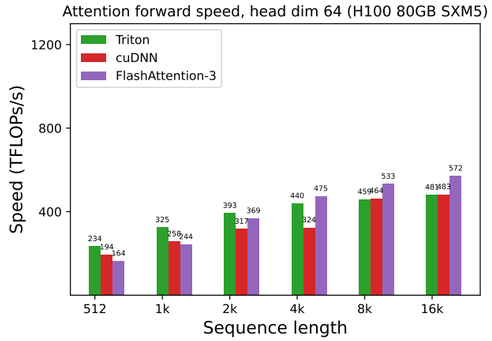
为降低数值波动，我们将GPU的时钟频率固定为1830MHz（该频率用于计算989 TFLOPs的FP16理论最大吞吐量）。我们重复执行这些基准测试100次，并取平均耗时。

## C.2 FP8AttentionFull结果

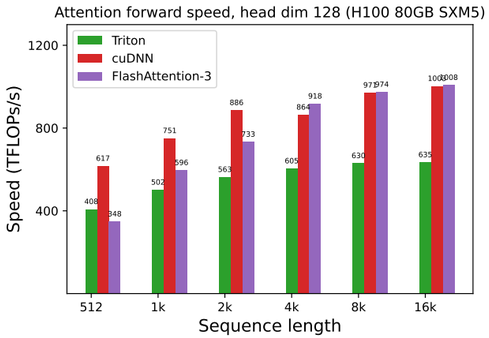
我们使用的序列长度为：512、1024、2048、4224、8448、16896。当序列长度为 ≥ 4k时，我们会将其调整为132（H100 SXM5架构中流多处理器的数量）的倍数，以避免波量化现象。



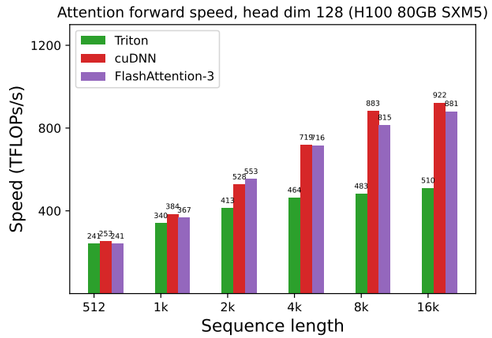

(a) Forward, without causal mask, head dim 64



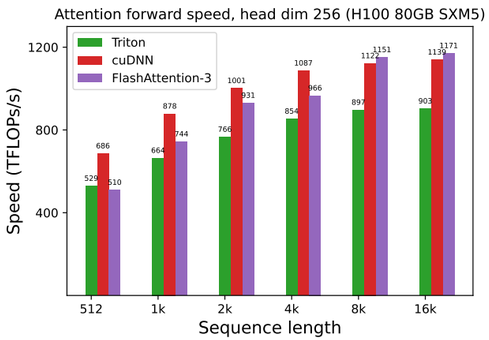

(b) Forward, with causal mask, head dim 64



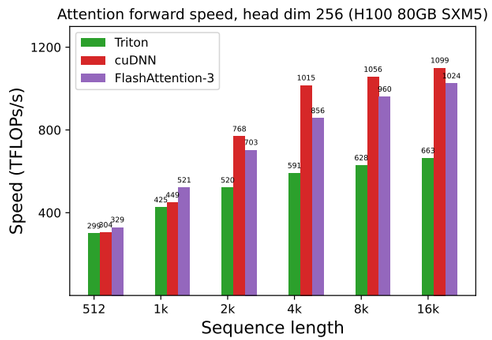

(c) Forward, without causal mask, head dim 128




(d) Forward, with causal mask, head dim 128



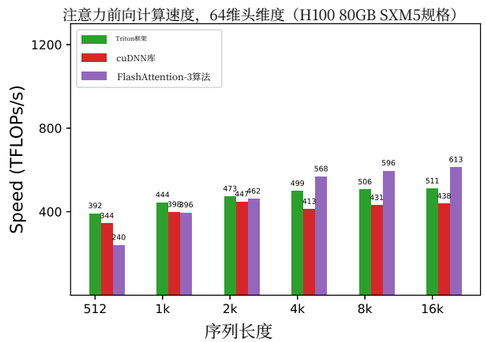

(e) Forward, without causal mask, head dim 256



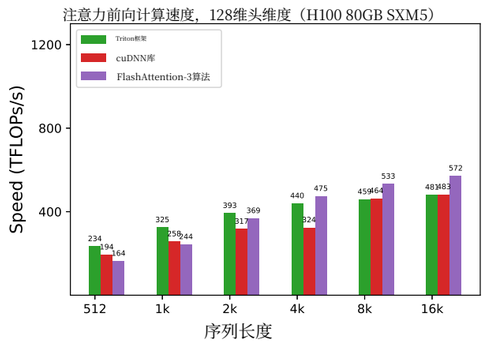

(f) Forward, with causal mask, head dim 256

Figure 9: Attention forward speed (FP8) on H100 GPU

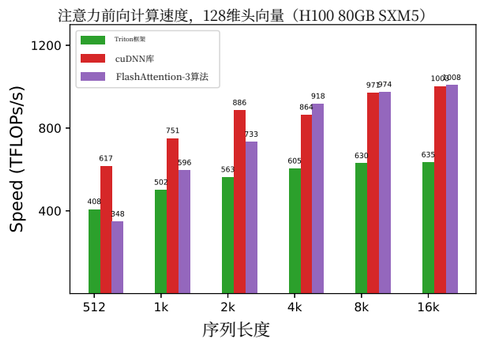




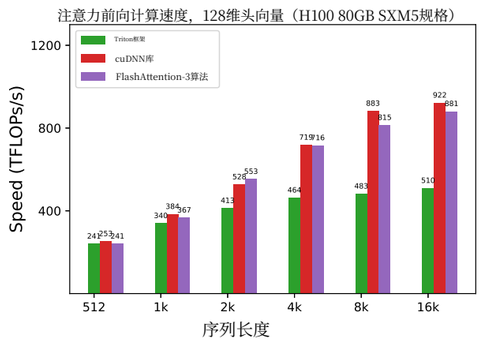

(a) 无因果掩码、注意力头维度为64的前向传播



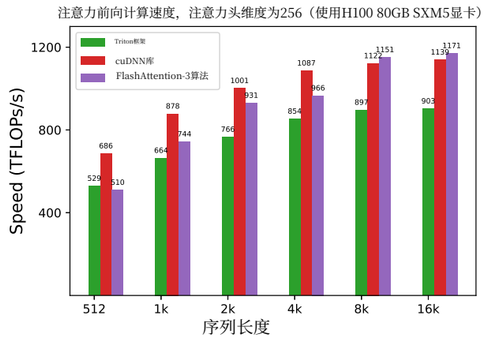

(b) 有因果掩码、注意力头维度为64的前向传播



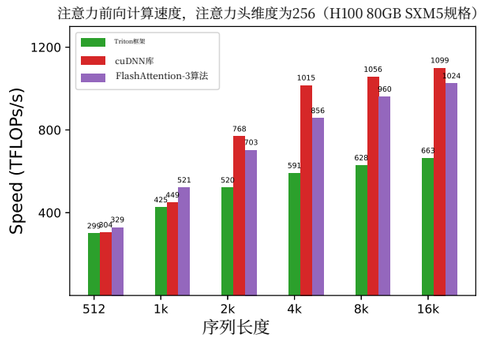

(c) 无因果掩码、注意力头维度为128的前向传播




(d) 有因果掩码、注意力头维度为128的前向传播




(e) 无因果掩码、注意力头维度为256的前向传播




(f) 有因果掩码、注意力头维度为256的前向传播

图9：H100显卡上采用FP8格式时的注意力前向计算速度